

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO NỘI DUNG ĐỒ ÁN
ĐỀ TÀI: THIẾT KẾ VÀ XÂY DỰNG TRÒ CHƠI NẤU ĂN
3D

Giảng viên hướng dẫn : TS. Nguyễn Việt Hưng

Lớp : CNCNTT4_K63

STT	Sinh viên thực hiện	Mã sinh viên
1	Nguyễn Mạnh Hùng	221220848

Hà Nội – 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO NỘI DUNG ĐỒ ÁN
ĐỀ TÀI: THIẾT KẾ VÀ XÂY DỰNG TRÒ CHƠI NẤU ĂN
3D

Giảng viên hướng dẫn : TS. Nguyễn Việt Hưng

Lớp : CNCNTT4_K63

STT	Sinh viên thực hiện	Mã sinh viên
1	Nguyễn Mạnh Hùng	221220848

Hà Nội – 2026

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án này, em đã nhận được rất nhiều sự giúp đỡ, hướng dẫn và động viên từ quý thầy cô, gia đình và bạn bè. Em xin gửi lời cảm ơn chân thành và sâu sắc đến tất cả những người đã đồng hành cùng em trong quá trình học tập và hoàn thành đề tài.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến **Thầy Nguyễn Việt Hưng**, giảng viên hướng dẫn đã tận tình chỉ bảo, định hướng và hỗ trợ em trong suốt quá trình thực hiện đồ án. Những góp ý chuyên môn, kinh nghiệm thực tiễn cũng như sự động viên của thầy đã giúp em từng bước hoàn thiện đề tài một cách tốt nhất.

Em xin chân thành cảm ơn **Thầy Hoàng Văn Thông**, Trưởng *khoa Công nghệ Thông tin*, cùng toàn thể các thầy cô trong khoa đã tận tâm giảng dạy, truyền đạt những kiến thức quý báu và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập và nghiên cứu tại trường. Những kiến thức nền tảng và chuyên môn mà quý thầy cô trang bị chính là hành trang quan trọng giúp em hoàn thành đồ án này.

Em cũng xin gửi lời cảm ơn đến gia đình, người thân và bạn bè đã luôn động viên, hỗ trợ và tạo điều kiện thuận lợi để em có thể tập trung học tập và hoàn thành tốt nhiệm vụ được giao.

Mặc dù đã cố gắng hết sức trong quá trình thực hiện, nhưng do thời gian, kinh nghiệm và kiến thức thực tế còn hạn chế nên đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp, nhận xét từ quý thầy cô để có thể tiếp tục hoàn thiện kiến thức và kỹ năng của bản thân trong tương lai.

Cuối cùng, em xin kính chúc quý thầy cô sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy.

Em xin chân thành cảm ơn!

LỜI CAM ĐOAN

Em xin cam đoan đồ án tốt nghiệp với đề tài “**Thiết kế và xây dựng trò chơi nấu ăn 3D**” là nghiên cứu được thực hiện bởi cá nhân em dưới sự giám sát của Thầy Nguyễn Việt Hưng.

Các số liệu, kết quả thực nghiệm, hình ảnh giao diện và toàn bộ nội dung mã nguồn được trình bày trong đồ án này là trung thực, do em tự tìm hiểu, thiết kết và lập trình trực tiếp dưới sự giúp đỡ của các công cụ hiện đại, hoàn toàn không sao chép từ bất kỳ công trình nghiên cứu hay đồ án của tác giả nào khác. Mọi nguồn tài liệu tham khảo, các thư viện mã nguồn mở, các package hỗ trợ đều được trích dẫn nguồn rõ ràng. Nếu có bất kỳ sự sao chép trái phép hay gian lận nào trong nội dung báo cáo này, em xin chịu hoàn toàn trách nhiệm trước Hội đồng bảo vệ tốt nghiệp đồ án, Khoa công nghệ thông tin cũng như các quy định kỷ luật của trường Đại học Giao Thông Vận Tải.

Hà Nội, tháng 6 năm 2026

Sinh viên thực hiện

Nguyễn Mạnh Hùng

LỜI NÓI ĐẦU

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ thông tin đã tạo điều kiện cho ngành công nghiệp trò chơi điện tử phát triển nhanh chóng và trở thành một lĩnh vực có sức ảnh hưởng lớn trong đời sống giải trí hiện đại. Không chỉ mang tính giải trí, các trò chơi điện tử còn là môi trường ứng dụng tổng hợp của nhiều lĩnh vực như lập trình, đồ họa máy tính, trí tuệ nhân tạo và công nghệ mạng.

Trong số các thể loại trò chơi hiện nay, các trò chơi mô phỏng kết hợp yếu tố hợp tác giữa nhiều người chơi đang ngày càng được quan tâm nhờ khả năng tạo ra trải nghiệm tương tác cao và mang tính giải trí hấp dẫn. Đặc biệt, các trò chơi mô phỏng hoạt động nấu ăn trong môi trường nhà bếp đã thu hút được nhiều người chơi bởi lối chơi đơn giản nhưng đòi hỏi sự phối hợp và phản xạ nhanh giữa các người chơi.

Xuất phát từ xu hướng đó, đề tài “**Thiết kế và xây dựng trò chơi nấu ăn 3D**” được thực hiện với mục tiêu xây dựng một trò chơi mô phỏng hoạt động nấu ăn trong môi trường ba chiều, cho phép người chơi thực hiện các thao tác như thu thập nguyên liệu, chế biến món ăn và phục vụ theo yêu cầu. Bên cạnh chế độ chơi đơn, trò chơi còn được phát triển thêm chế độ chơi hợp tác trực tuyến, giúp nhiều người chơi có thể cùng tham gia và phối hợp hoàn thành nhiệm vụ trong cùng một màn chơi.

Thông qua việc thực hiện đề tài này, sinh viên có cơ hội vận dụng các kiến thức đã học về lập trình, phát triển trò chơi và công nghệ mạng vào việc xây dựng một sản phẩm thực tế. Đồng thời, quá trình nghiên cứu và triển khai đề tài cũng giúp nâng cao kỹ năng thiết kế hệ thống, giải quyết vấn đề và phát triển phần mềm trong môi trường thực tế.

MỤC LỤC

LỜI CẢM ƠN	3
LỜI CAM ĐOAN	4
LỜI NÓI ĐẦU	5
MỤC LỤC	6
DANH MỤC HÌNH VẼ	9
DANH MỤC BẢNG BIỂU	10
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI	11
1.1. Giới thiệu bài toán	11
1.1.1. Giới thiệu chung	11
1.1.2. Phạm vi sơ bộ và mục tiêu đề tài	11
1.1.3. Lý do chọn đề tài	12
1.2. Phát biểu bài toán	12
1.2.1. Yêu cầu cốt lõi của hệ thống	12
1.2.2. Yêu cầu chức năng.....	13
1.2.3. Yêu cầu phi chức năng	14
1.2.4. Mục tiêu chất lượng.....	14
1.3. Khảo sát và phân tích các sản phẩm tương tự	15
1.3.1. Overcooked 2 (Ghost Town Games, 2018)	15
1.3.2. PlateUp! (Yogscat Games, 2022).....	15
1.3.3. Bảng so sánh tổng hợp.....	15
1.3.4. Kết luận khảo sát	16
1.4. Đóng góp của đề tài	16
1.5. Giới thiệu về công cụ Unity Engine	18
1.5.1. Tổng quan về Unity	18
1.5.2. Unity trong thiết kế trò chơi nấu ăn 3D.....	18
1.6. Ngôn ngữ C#.....	18
1.7. Giới thiệu về IDE.....	19
1.8. Giải pháp lập trình mạng – Photon Fusion	19

1.8.1.	Tổng quan về Photon Fusion	19
1.8.2.	Kiến trúc Client-Server và cơ chế đồng bộ trạng thái	20
1.9.	Các mẫu thiết kế (Design Patterns) áp dụng	20
1.9.1.	Singleton Pattern.....	20
1.9.2.	Observer Pattern	21
1.9.3.	Object Pooling Pattern.....	21
1.9.4.	Finite State Machine (FSM)	22
1.10.	Prefab trong Unity	23
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG		24
2.1.	Phân tích yêu cầu hệ thống	24
2.1.1.	Các tác nhân (Actor) trong hệ thống	24
2.1.2.	Đặc tả use case chính.....	26
2.2.	Kiến trúc của hệ thống.....	30
2.3.	Thiết kế kiến trúc hệ thống	33
2.3.1.	Mô tả kiến trúc tổng thể.....	33
2.3.2.	Thiết kế lớp Counter	33
2.4.	Thiết kế cơ sở dữ liệu game.....	33
2.5.	Biểu đồ tuần tự.....	33
CHƯƠNG 3: CÀI ĐẶT, KIỂM THỬ VÀ ĐÁNH GIÁ		40
3.1.	Tổng quan quá trình triển khai	40
3.2.	Triển khai quản lý trò chơi (gameplay) và xử lý đầu vào (input).....	40
3.2.1.	Mục tiêu thiết kế	40
3.2.2.	Kiến trúc tổng thể	40
3.2.3.	Xử lý input người chơi.....	41
3.3.	Triển khai logic di chuyển	41
3.3.1.	Vai trò của lớp GameLogic.....	41
3.3.2.	Luồng xử lý hành động của trò chơi.....	41
3.3.3.	Hệ thống chế biến món ăn	42
3.4.	Triển khai hệ thống bản đồ của mỗi cấp độ.....	42
3.5.	Triển khai các thực thể trong trò chơi.....	42

3.5.1.	Tổng quan thiết kế thực thể	42
3.5.2.	Nhân vật của người chơi.....	43
3.5.3.	Nguyên liệu và món ăn	43
3.5.4.	Dụng cụ bếp.....	43
3.6.	Triển khai hiển thị và tài nguyên	43
3.7.	Triển khai hệ thống mạng Multiplayer với Photon Fusion.....	44
3.7.1.	Khởi tạo kết nối và quản lý phòng chơi.....	44
3.7.2.	Đồng bộ trạng thái nhân vật và input người chơi	45
3.7.3.	Quản lý quyền kiểm soát trạng thái (State Authority)	45
3.7.4.	Chuyển dữ liệu từ Lobby sang GameScene	46
3.8.	Triển khai hệ thống Level Editor.....	46
3.8.1.	Kiến trúc tổng thể của Level Editor.....	46
3.8.2.	Tuần tự hóa và lưu trữ dữ liệu	46
3.9.	Kết quả đạt được.....	47
3.10.	Kiểm thử chức năng.....	53
3.11.	Đánh giá hệ thống	53
3.12.	Hạn chế	54
3.13.	Kết luận.....	55
TÀI LIỆU THAM KHẢO.....		56

DANH MỤC HÌNH VẼ

Hình 1.1: Hình so sánh Singleton với Normal Class	20
Hình 1.2: Hình của mẫu thiết kế Observer	21
Hình 1.3: Hình của mẫu thiết kế Object Pooling	22
Hình 1.4: Hình mẫu thiết kế Finite State Machine	23
Hình 2.1: Sơ đồ ca sử dụng	25
Hình 2.2: Sơ đồ lớp	32
Hình 2.3: Sơ đồ tuần tự tạo phòng chơi và bắt đầu ván chơi	35
Hình 2.4: Sơ đồ tuần tự nấu và giao món ăn	37
Hình 2.5: Sơ đồ hành động cho gameplay chính	39
Hình 3.1: Hình ảnh của Scene Level Design	48
Hình 3.2: Hình ảnh lưu level vào Prefab	49
Hình 3.3: Hình ảnh của Screen MainMenu	49
Hình 3.4: Hình ảnh của Screen khi chọn Multiplayer	50
Hình 3.5: Hình ảnh của Sreenk khi tạo phòng bằng Photon Fushion	50
Hình 3.6: Hình ảnh của sảnh chờ vào phòng	51
Hình 3.7: Hình ảnh khi vào phòng	51
Hình 3.8: Hình ảnh khi vào phòng	52
Hình 3.9: Hình ảnh khi vào trò chơi	52
Hình 3.10: Hình ảnh khi vào trò chơi multiplayer	53

DANH MỤC BẢNG BIỂU

Bảng 1.1: Danh sách yêu cầu chức năng của hệ thống.....	14
Bảng 1.3: Bảng so sánh tổng hợp	16
Bảng 2.1: Bảng use case Tạo và tham gia phòng chơi	26
Bảng 2.2: Bảng use case Sơ chế nguyên liệu	27
Bảng 2.3: Bảng use case Giao món ăn và tính điểm	28
Bảng 2.4: Bảng use case Thiết kế và lưu màn chơi.....	28
Bảng 2.5: Bảng use case Nấu món ăn.....	29
Bảng 2.6: Bảng use case Chọn màn và chơi.....	30
Bảng 2.7: Bảng use case Chọn và khởi chạy màn chơi.....	30

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1. Giới thiệu bài toán

1.1.1. Giới thiệu chung

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin, ngành công nghiệp trò chơi điện tử đã đạt được nhiều bước tiến đáng kể. Các trò chơi không chỉ dừng lại ở mục đích giải trí mà còn trở thành một lĩnh vực kết hợp nhiều công nghệ hiện đại như lập trình, đồ họa máy tính, vật lý mô phỏng và công nghệ mạng. Đặc biệt, các trò chơi mô phỏng kết hợp yếu tố hợp tác giữa nhiều người chơi đang ngày càng được quan tâm nhờ khả năng tạo ra trải nghiệm tương tác và tính giải trí cao.

Trong số đó, các trò chơi mô phỏng hoạt động nấu ăn trong môi trường nhà bếp đang thu hút được nhiều người chơi bởi lối chơi đơn giản nhưng đòi hỏi sự phối hợp, phản xạ nhanh và khả năng xử lý tình huống. Người chơi phải thực hiện nhiều thao tác như thu thập nguyên liệu, chế biến món ăn và phục vụ theo yêu cầu trong khoảng thời gian giới hạn.

Xuất phát từ những xu hướng trên, đề tài “Thiết kế và phát triển trò chơi nấu ăn 3D” được thực hiện nhằm xây dựng một trò chơi mô phỏng hoạt động nấu ăn trong môi trường ba chiều. Trò chơi cho phép người chơi điều khiển nhân vật di chuyển trong không gian bếp, tương tác với các nguyên liệu và dụng cụ nấu ăn, từ đó hoàn thành các món ăn theo yêu cầu của trò chơi. Bên cạnh đó, trò chơi còn được thiết kế với chế độ chơi đơn và chế độ chơi hợp tác trực tuyến, giúp nhiều người chơi có thể cùng tham gia và phối hợp hoàn thành nhiệm vụ.

Thông qua đề tài này, sinh viên có thể áp dụng các kiến thức đã học về lập trình, thiết kế hệ thống và phát triển trò chơi để xây dựng một sản phẩm thực tế có tính ứng dụng và giải trí.

1.1.2. Phạm vi sơ bộ và mục tiêu đề tài

Đối tượng người dùng: Người chơi yêu thích các trò chơi mô phỏng và trò chơi mang tính hợp tác.

Nền tảng: Máy tính cá nhân (hệ điều hành Windows).

Chức năng chính trong phạm vi đề tài:

- Điều khiển nhân vật di chuyển trong môi trường bếp 3D.
- Tương tác với các đối tượng trong trò chơi như nguyên liệu, dụng cụ nấu ăn và khu vực chế biến.
- Thực hiện các bước chế biến món ăn theo yêu cầu của trò chơi.
- Hoàn thành các nhiệm vụ nấu ăn trong thời gian giới hạn để đạt điểm số.
- Hỗ trợ chế độ chơi đơn và chế độ chơi hợp tác trực tuyến giữa nhiều người chơi.

- Hiện thị giao diện trò chơi, điểm số và trạng thái nhiệm vụ.

Mục tiêu đề tài

- Xây dựng một trò chơi nấu ăn 3D có thể hoạt động ổn định trên máy tính cá nhân.
- Thiết kế hệ thống điều khiển nhân vật và cơ chế tương tác với các đối tượng trong môi trường trò chơi.
- Phát triển lối chơi đơn giản, dễ tiếp cận nhưng vẫn mang tính thử thách nhằm tạo trải nghiệm thú vị cho người chơi.
- Triển khai chế độ chơi hợp tác trực tuyến cho phép nhiều người chơi cùng tham gia trong một màn chơi và phối hợp thực hiện nhiệm vụ.
- Áp dụng các kiến thức đã học về lập trình, thiết kế phần mềm và phát triển trò chơi để xây dựng một sản phẩm hoàn chỉnh.

1.1.3. Lý do chọn đề tài

Trong bối cảnh ngành công nghiệp trò chơi điện tử ngày càng phát triển, các trò chơi mang tính mô phỏng và hợp tác giữa nhiều người chơi đang ngày càng được ưa chuộng nhờ khả năng tạo ra trải nghiệm giải trí hấp dẫn và tăng tính tương tác giữa người chơi. Đặc biệt, các trò chơi mô phỏng hoạt động nấu ăn trong môi trường nhà bếp mang lại lối chơi thú vị, đòi hỏi người chơi phải phối hợp nhịp nhàng để hoàn thành các nhiệm vụ trong thời gian giới hạn.

Xuất phát từ những lý do đó, đề tài **“Thiết kế và phát triển trò chơi nấu ăn 3D”** được lựa chọn nhằm xây dựng một trò chơi mô phỏng hoạt động nấu ăn trong môi trường ba chiều. Thông qua việc phát triển trò chơi này, sinh viên có cơ hội áp dụng kiến thức đã học về lập trình, thiết kế hệ thống và phát triển trò chơi vào một sản phẩm thực tế.

Bên cạnh đó, việc xây dựng trò chơi còn giúp rèn luyện kỹ năng lập trình, tư duy thiết kế hệ thống và khả năng giải quyết các vấn đề phát sinh trong quá trình phát triển phần mềm. Đồng thời, sản phẩm tạo ra cũng có thể được tiếp tục phát triển và mở rộng thêm nhiều tính năng trong tương lai.

1.2. Phát biểu bài toán

1.2.1. Yêu cầu cốt lõi của hệ thống

Hệ thống cần cung cấp một trải nghiệm chơi hấp dẫn, mô phỏng hoạt động nấu ăn trong môi trường nhà bếp 3D. Người chơi sẽ điều khiển nhân vật thực hiện các thao tác như thu thập nguyên liệu, chế biến món ăn và phục vụ theo yêu cầu trong khoảng thời gian giới hạn. Các cơ chế chính của trò chơi cần được xây dựng rõ ràng nhằm đảm bảo sự tương tác mượt mà giữa người chơi và môi trường.

- **Cơ chế điều khiển và di chuyển nhân vật:** Hệ thống cần cho phép người chơi điều khiển nhân vật di chuyển linh hoạt trong không gian bếp 3D. Việc điều khiển phải đơn giản, trực quan và có phản hồi nhanh khi người chơi thao tác.

- **Tương tác với các đối tượng trong môi trường bếp:** Người chơi có thể tương tác với nhiều đối tượng khác nhau như nguyên liệu, bàn chế biến, bếp nấu và khu vực phục vụ. Hệ thống cần quản lý các hành động như nhặt nguyên liệu, đặt nguyên liệu lên bàn chế biến, thực hiện các bước chế biến và phục vụ món ăn.
- **Hệ thống nhiệm vụ và chế biến món ăn:** Trò chơi cần xây dựng cơ chế tạo yêu cầu món ăn và kiểm tra các bước chế biến của người chơi. Người chơi phải thực hiện đúng các bước chế biến theo yêu cầu để hoàn thành món ăn và nhận điểm thưởng.
- **Chế độ chơi hợp tác nhiều người chơi:** Hệ thống cần hỗ trợ chế độ chơi hợp tác trực tuyến cho phép nhiều người chơi tham gia cùng một màn chơi. Các trạng thái quan trọng như vị trí nhân vật, hành động và tiến trình nhiệm vụ cần được đồng bộ giữa các người chơi.
- **Quản lý tiến trình và điểm số:** Hệ thống cần theo dõi điểm số, trạng thái nhiệm vụ và tiến trình của màn chơi. Khi hoàn thành các yêu cầu trong màn chơi, người chơi sẽ nhận điểm và có thể chuyển sang các màn chơi tiếp theo.

1.2.2. Yêu cầu chức năng

Hệ thống cần cung cấp một trải nghiệm chơi hấp dẫn, mô phỏng hoạt động nấu ăn trong môi trường nhà bếp 3D. Người chơi sẽ điều khiển nhân vật thực hiện các thao tác như thu thập nguyên liệu, chế biến món ăn và phục vụ theo yêu cầu trong khoảng thời gian giới hạn. Các yêu cầu chức năng cụ thể được liệt kê trong bảng dưới đây.

Mã YC	Tên chức năng	Mô tả	Độ ưu tiên
F-01	Điều khiển nhân vật	Người chơi di chuyển nhân vật trong không gian bếp 3D bằng bàn phím (WASD). Nhân vật phản hồi tức thì, không trễ nhập liệu.	Cao
F-02	Tương tác với môi trường	Nhân vật có thể nhặt, đặt, và truyền nguyên liệu giữa các bàn bếp. Phạm vi tương tác được xác định bằng vùng va chạm của Counter.	Cao
F-03	Sơ chế nguyên liệu	Người chơi giữ phím tương tác tại CuttingCounter để tăng cuttingProgress. Khi đạt 100%, nguyên liệu đổi thành dạng đã cắt.	Cao
F-04	Nấu ăn (FSM)	StoveCounter quản lý 4 trạng thái: Idle → Frying → Fried → Burned. Hệ thống cảnh báo hình ảnh khi sắp cháy.	Cao
F-05	Giao món & tính	DeliveryCounter kiểm tra nội dung	Cao

	điểm	đĩa so với danh sách đơn hàng hiện tại. Giao đúng: cộng điểm và phát sự kiện OnRecipeSuccess.	
F-06	Chế độ Multiplayer	Hỗ trợ 2 người chơi kết nối trực tuyến qua Photon Fusion. Tất cả trạng thái gameplay được đồng bộ thời gian thực.	Cao
F-07	Lobby & quản lý phòng	Host tạo phòng với tên tùy chọn; Client tham gia bằng tên phòng. Người chơi chọn màu nhân vật tại Lobby trước khi bắt đầu.	Trung bình
F-08	Level Editor	Host thiết kế bố cục bếp bằng cách đặt Counter lên lưới tọa độ. Dữ liệu được lưu/tải dưới dạng Prefab trong Unity.	Trung bình
F-09	Bộ đếm thời gian & điểm số	Màn chơi có thời gian giới hạn. Điểm số cập nhật real-time trên HUD và đồng bộ về tất cả client trong chế độ multiplayer.	Trung bình

Bảng 1.1: Danh sách yêu cầu chức năng của hệ thống

1.2.3. Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, trò chơi cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo trải nghiệm tốt cho người chơi trên nhiều cấu hình máy tính khác nhau. Bảng 1.2: Danh sách yêu cầu phi chức năng

1.2.4. Mục tiêu chất lượng

Bên cạnh các yêu cầu chức năng, trò chơi cũng cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo trải nghiệm tốt cho người chơi.

- **Hiệu năng:** Trò chơi cần hoạt động ổn định với tốc độ khung hình mượt mà và đảm bảo phản hồi thao tác nhanh để người chơi có trải nghiệm tốt.
- **Thân thiện với người dùng:** Giao diện trò chơi cần được thiết kế trực quan, dễ hiểu và cung cấp hướng dẫn cơ bản giúp người chơi nhanh chóng làm quen với cơ chế chơi.
- **Tính ổn định:** Hệ thống cần hạn chế lỗi trong quá trình chơi, đảm bảo các chức năng chính như điều khiển nhân vật, tương tác với môi trường và chế biến món ăn hoạt động chính xác.
- **Khả năng mở rộng:** Cấu trúc hệ thống cần được thiết kế linh hoạt để có thể dễ dàng bổ sung thêm các món ăn mới, nguyên liệu, dụng cụ nấu ăn hoặc các màn chơi mới trong tương lai.

Phần phát biểu bài toán này nhằm xác định rõ những yêu cầu và mục tiêu chính mà hệ thống cần đáp ứng để xây dựng thành công trò chơi nấu ăn 3D lấy cảm hứng từ trò chơi Overcooked, đồng thời đảm bảo sản phẩm có tính ổn định và khả năng phát triển thêm trong tương lai.

1.3. Khảo sát và phân tích các sản phẩm tương tự

Trước khi tiến hành xây dựng trò chơi, nhóm đã tiến hành khảo sát một số tựa game có cùng thể loại mô phỏng nấu ăn đang phổ biến trên thị trường nhằm rút ra những điểm mạnh, điểm yếu và định hướng phát triển phù hợp cho đề tài.

1.3.1. Overcooked 2 (Ghost Town Games, 2018)

Tổng quan: Overcooked 2 là tựa game nấu ăn hợp tác nổi tiếng nhất hiện nay, hỗ trợ chế độ chơi online multiplayer cho tối đa 4 người. Điểm nổi bật là thiết kế màn chơi sáng tạo với các bản đồ bếp di động, yêu cầu người chơi phải phối hợp liên tục trong môi trường thay đổi liên tục. Game sử dụng cơ chế ném (throw) nguyên liệu giữa các bếp, tăng tính hỗn loạn và thách thức.

Điểm mạnh: Gameplay đơn giản, dễ tiếp cận; thiết kế màn chơi phong phú; hỗ trợ đến 4 người chơi trực tuyến; có chế độ chơi cạnh tranh (Versus Mode).

Điểm yếu: Hệ thống mạng đôi khi bị trễ (lag) do phụ thuộc vào kết nối P2P; không có công cụ cho phép người dùng tự thiết kế màn chơi riêng; không hỗ trợ lưu trữ và chia sẻ màn chơi tùy chỉnh.

1.3.2. PlateUp! (Yogscat Games, 2022)

Tổng quan: PlateUp! là thế hệ tiếp theo của thể loại game nấu ăn mô phỏng, bổ sung thêm yếu tố xây dựng nhà hàng và tự động hóa dây chuyền nấu ăn. Game cho phép người chơi đặt băng chuyền, máy móc tự động hóa các bước chế biến. Hệ thống mạng sử dụng Steam Networking, đảm bảo độ trễ thấp và kết nối ổn định.

Điểm mạnh: Chiều sâu gameplay vượt trội với hệ thống tự động hóa; mạng ổn định, ít lag; đồ họa 3D chi tiết và sinh động.

Điểm yếu: Độ phức tạp cao khiến người chơi mới khó tiếp cận; yêu cầu cấu hình máy tính tương đối cao; không có Level Editor cho phép người dùng tạo bố cục bếp tùy chỉnh.

1.3.3. Bảng so sánh tổng hợp

Tiêu chí	Overcooked 2	PlateUp!	Đề tài này
Đồ họa	3D đơn giản	3D chi tiết	3D Unity (tự phát triển)
Chế độ multiplayer	Online P2P, tối đa 4 người	Online Steam, tối đa 4 người	Online Photon Fusion, 2 người

Chất lượng kết nối	Trung bình (P2P dễ lag)	Tốt (Steam relay)	Tốt (Client-Side Prediction)
Level Editor	Không có	Giới hạn	Có – lưu/tải Prefab tùy chỉnh
Khả năng mở rộng	Thấp (hardcode)	Trung bình	Cao (ScriptableObject + OOP)
Design Pattern	Không công bố	Không công bố	Singleton, Observer, FSM, Pooling
Độ phức tạp gameplay	Thấp – trung bình	Cao	Trung bình – dễ tiếp cận
Yêu cầu phần cứng	Thấp	Cao	Thấp – trung bình

Bảng 1.3: Bảng so sánh tổng hợp

1.3.4. Kết luận khảo sát

Qua quá trình phân tích, có thể thấy các sản phẩm hiện có đều thiếu một công cụ Level Editor linh hoạt cho phép người dùng tự thiết kế màn chơi và lưu trữ dưới định dạng dữ liệu mở. Bên cạnh đó, cơ chế đồng bộ mạng của các game này chưa được tối ưu cho môi trường kết nối có độ trễ cao. Đây chính là hai điểm mà đề tài tập trung cải thiện: tích hợp hệ thống Level Editor với Prefab serialization và sử dụng Photon Fusion với cơ chế Client-Side Prediction để giảm thiểu cảm giác trễ khi chơi mạng.

1.4. Đóng góp của đề tài

Bên cạnh việc xây dựng một trò chơi nấu ăn 3D theo phong cách Overcooked, đề tài còn tập trung nghiên cứu và triển khai các giải pháp kỹ thuật nhằm nâng cao khả năng mở rộng và tính thực tiễn của hệ thống.

Đóng góp đầu tiên của đề tài là xây dựng thành công hệ thống chơi mạng nhiều người chơi (Multiplayer) dựa trên Photon Fusion. Hệ thống cho phép nhiều người chơi cùng tham gia vào một phòng chơi và thực hiện các thao tác tương tác trong thời gian thực. Các trạng thái quan trọng của trò chơi được đồng bộ giữa Host và Client nhằm đảm bảo tính nhất quán của dữ liệu trong suốt quá trình chơi.

Đóng góp thứ hai là việc xây dựng hệ thống Level Editor cho phép người dùng tự thiết kế bố cục màn chơi theo nhu cầu. Người chơi có thể lựa chọn các loại bàn bếp, sắp xếp vị trí trong không gian và lưu lại màn chơi để sử dụng về sau. Đây là tính năng không xuất hiện trong trò chơi Overcooked gốc và góp phần tăng khả năng mở rộng nội dung của trò chơi.

Đóng góp thứ ba là áp dụng cơ chế tuần tự hóa dữ liệu bằng Prefab để lưu trữ thông tin màn chơi. Việc lưu trữ dữ liệu dưới dạng Prefab giúp quá trình tạo, chỉnh sửa và quản lý màn chơi trở nên linh hoạt hơn, đồng thời tạo tiền đề cho việc mở rộng hệ thống chia sẻ màn chơi trong tương lai.

Ngoài ra, đề tài còn áp dụng nhiều kỹ thuật phát triển phần mềm hiện đại như lập trình hướng đối tượng, Observer Pattern, Singleton Pattern, Object Pooling và Finite State Machine nhằm tăng khả năng bảo trì, mở rộng và tối ưu hiệu năng của hệ thống.

1.5. Giới thiệu về công cụ Unity Engine

1.5.1. Tổng quan về Unity

Unity là một nền tảng phát triển trò chơi (game engine) phổ biến được sử dụng rộng rãi trong việc xây dựng các trò chơi 2D và 3D cũng như các ứng dụng tương tác. Unity cung cấp một môi trường phát triển tích hợp với giao diện trực quan, cho phép nhà phát triển dễ dàng thiết kế cảnh, quản lý đối tượng và xây dựng logic cho trò chơi. Hệ thống kiến trúc dựa trên component giúp việc phát triển trở nên linh hoạt, khi mỗi đối tượng trong trò chơi có thể được gắn thêm các thành phần để mở rộng chức năng. Ngoài ra, Unity hỗ trợ lập trình bằng ngôn ngữ C#, cung cấp nhiều thư viện và công cụ hỗ trợ cho việc xử lý vật lý, animation, âm thanh và giao diện người dùng. Với khả năng triển khai trên nhiều nền tảng khác nhau như máy tính, thiết bị di động và web, cùng với cộng đồng người dùng lớn và nguồn tài liệu phong phú, Unity đã trở thành một trong những công cụ phát triển trò chơi được sử dụng phổ biến hiện nay.

1.5.2. Unity trong thiết kế trò chơi nấu ăn 3D

Unity cung cấp mô hình lập trình dựa trên kiến trúc thành phần (Component), cho phép các đối tượng trong trò chơi được xây dựng dưới dạng đối tượng trò chơi (GameObject) kết hợp với các tập lệnh được viết bằng ngôn ngữ C#. Cách tiếp cận này giúp việc phát triển trò chơi trở nên linh hoạt, khi mỗi đối tượng trong môi trường nhà bếp như nhân vật, nguyên liệu, dụng cụ nấu ăn hoặc khu vực chế biến có thể được gắn thêm các thành phần chức năng khác nhau để mở rộng khả năng hoạt động.

Phương pháp phát triển theo mô-đun này cũng phù hợp với việc thiết kế hệ thống trò chơi, trong đó các thành phần như xử lý điều khiển nhân vật (Input), logic trò chơi (Game Logic), quản lý nhiệm vụ và hệ thống tương tác được tách thành các phần riêng biệt. Điều này giúp việc phát triển, kiểm thử và mở rộng hệ thống trở nên thuận lợi hơn.

Ngoài ra, Unity còn cung cấp nhiều công cụ hỗ trợ mạnh mẽ cho việc phát triển trò chơi ba chiều (3D) như hệ thống vật lý (Physics), hệ thống chuyển động nhân vật (Animation), quản lý cảnh trò chơi (Scene) và xây dựng giao diện người dùng (User Interface – UI). Những công cụ này giúp việc thiết kế môi trường nhà bếp, xây dựng các đối tượng tương tác và triển khai cơ chế nấu ăn trong trò chơi trở nên trực quan và hiệu quả.

1.6. Ngôn ngữ C#

C# là một ngôn ngữ lập trình hiện đại do Microsoft phát triển, được thiết kế theo hướng đơn giản, an toàn và dễ sử dụng. C# hỗ trợ mạnh mẽ lập trình hướng đối tượng, có cú pháp rõ ràng, dễ đọc và cung cấp nhiều thư viện hỗ trợ cho quá trình phát triển phần mềm. Nhờ khả năng linh hoạt và hiệu suất tốt, C# được sử dụng rộng rãi trong

nhiều lĩnh vực như phát triển ứng dụng desktop, ứng dụng web và đặc biệt là trong phát triển trò chơi thông qua công cụ Unity.

C# hỗ trợ đầy đủ các đặc trưng của lập trình hướng đối tượng như đóng gói, kế thừa và đa hình. Trong đề tài “**Thiết kế và phát triển trò chơi nấu ăn 3D**”, C# được sử dụng để xây dựng các thành phần chính của hệ thống trò chơi như hệ thống điều khiển nhân vật, quản lý trạng thái trò chơi, quản lý nhiệm vụ và xử lý tương tác giữa nhân vật với các đối tượng trong môi trường nhà bếp.

Việc sử dụng C# theo phương pháp lập trình hướng đối tượng giúp mã nguồn của trò chơi có cấu trúc rõ ràng, dễ quản lý và thuận tiện cho việc bảo trì. Đồng thời, cách tổ chức này cũng giúp việc mở rộng thêm các chức năng mới như bổ sung món ăn, nguyên liệu hoặc các cơ chế chơi mới trong tương lai trở nên dễ dàng hơn.

1.7. Giới thiệu về IDE

JetBrains Rider là một IDE mạnh mẽ, chủ yếu dành cho phát triển .NET, Unity và C#, nổi bật với khả năng phân tích code sâu, refactor thông minh và tích hợp tốt với hệ sinh thái JetBrains. Rider giúp lập trình viên phát hiện lỗi sớm, tối ưu cấu trúc mã và nâng cao hiệu suất làm việc trong các dự án quy mô vừa và lớn.

Visual Studio Code (VS Code) là một trình soạn thảo mã nguồn nhẹ, đa nền tảng, được Microsoft phát triển. VS Code nổi bật nhờ tốc độ nhanh, giao diện đơn giản và hệ sinh thái extension phong phú, hỗ trợ nhiều ngôn ngữ lập trình khác nhau. Nhờ tính linh hoạt cao, VS Code phù hợp cho nhiều mục đích phát triển, từ ứng dụng web, backend đến scripting và DevOps.

Các công cụ IDE trên hỗ trợ hiệu quả cho quá trình phát triển và quản lý mã nguồn C# trong Unity, giúp nhóm kiểm soát chất lượng code và tăng năng suất làm việc trong suốt quá trình thực hiện dự án.

1.8. Giải pháp lập trình mạng – Photon Fusion

1.8.1. Tổng quan về Photon Fusion

Photon Fusion là một giải pháp lập trình mạng (networking framework) thế hệ mới do Photon Engine phát triển, chuyên dùng cho các trò chơi multiplayer thời gian thực trên nền tảng Unity. So với các giải pháp trước đó như Photon PUN2, Mirror hay Netcode for GameObjects (NGO), Photon Fusion được đánh giá là tối ưu hơn về mặt hiệu năng và khả năng đồng bộ trạng thái nhờ kiến trúc tick-based deterministic.

Photon Fusion cung cấp hai chế độ hoạt động chính: Shared Mode (mô hình ngang hàng, không cần host) và Host Mode (mô hình có host kiểm soát toàn bộ trạng thái game). Trong đề tài này, Host Mode được lựa chọn vì phù hợp hơn với tính chất gameplay cần kiểm soát chặt chẽ trạng thái các đối tượng trong bếp.

1.8.2. Kiến trúc Client-Server và cơ chế đồng bộ trạng thái

Photon Fusion sử dụng kiến trúc Server-Authoritative, trong đó host (máy chủ) là nơi duy nhất có quyền thay đổi trạng thái game. Các client chỉ gửi input lên host và nhận lại trạng thái đã được xác nhận. Cơ chế này đảm bảo tính nhất quán của trò chơi, tránh tình trạng cheating và xung đột dữ liệu giữa các người chơi.

Để giảm thiểu cảm giác trễ cho người dùng, Photon Fusion tích hợp cơ chế Client-Side Prediction: client tự dự đoán kết quả hành động của người chơi ngay lập tức mà không cần chờ xác nhận từ host. Nếu kết quả dự đoán khác với trạng thái nhận về từ host, hệ thống sẽ tự động hiệu chỉnh lại (Reconciliation), đảm bảo trải nghiệm mượt mà.

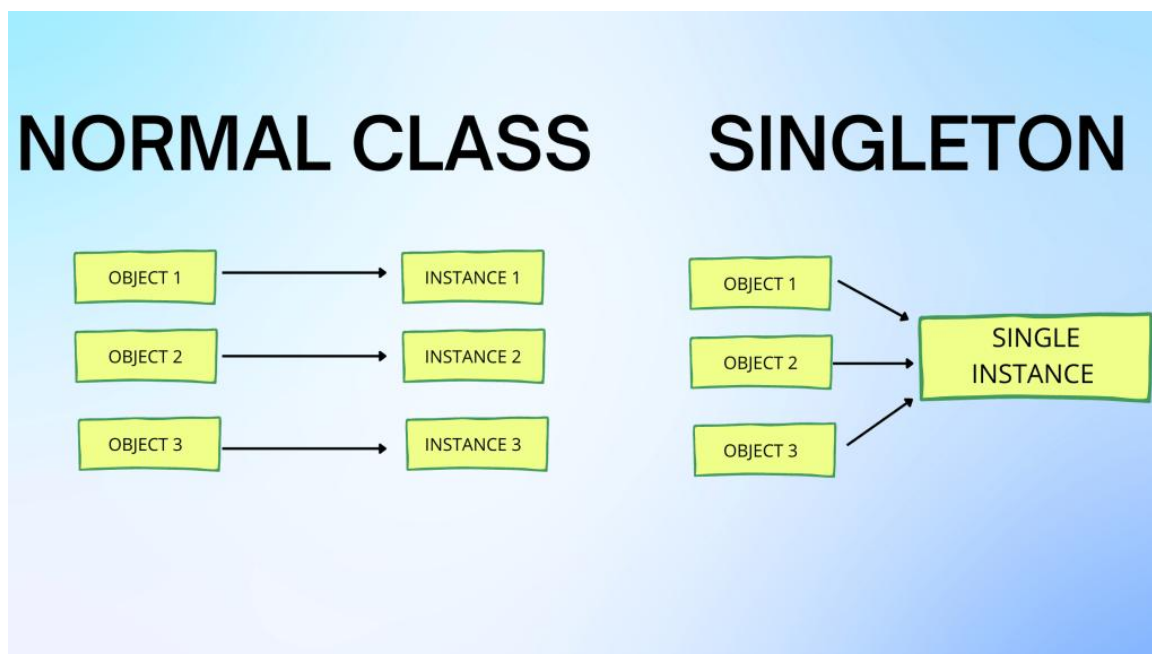
Các biến trạng thái cần đồng bộ giữa host và client được khai báo bằng thuộc tính [Networked], ví dụ như vị trí nhân vật, trạng thái nấu của bếp hay điểm số. Photon Fusion sẽ tự động truyền các biến này từ host về tất cả client sau mỗi tick mạng.

1.9. Các mẫu thiết kế (Design Patterns) áp dụng

1.9.1. Singleton Pattern

Singleton Pattern đảm bảo rằng một lớp chỉ có duy nhất một thực thể tồn tại trong suốt vòng đời ứng dụng, và cung cấp một điểm truy cập toàn cục đến thực thể đó. Trong đề tài này, Singleton được triển khai dưới dạng lớp generic abstract kế thừa MonoBehaviour, được áp dụng cho các manager toàn cục như MessageManager, PoolManager, UIManager và FusionNetworkRunner.

Việc sử dụng Singleton cho các manager giúp bất kỳ script nào trong game cũng có thể truy cập trực tiếp thông qua thuộc tính Instance mà không cần tìm kiếm đối tượng trong scene, giảm chi phí tính toán và tăng tính nhất quán.

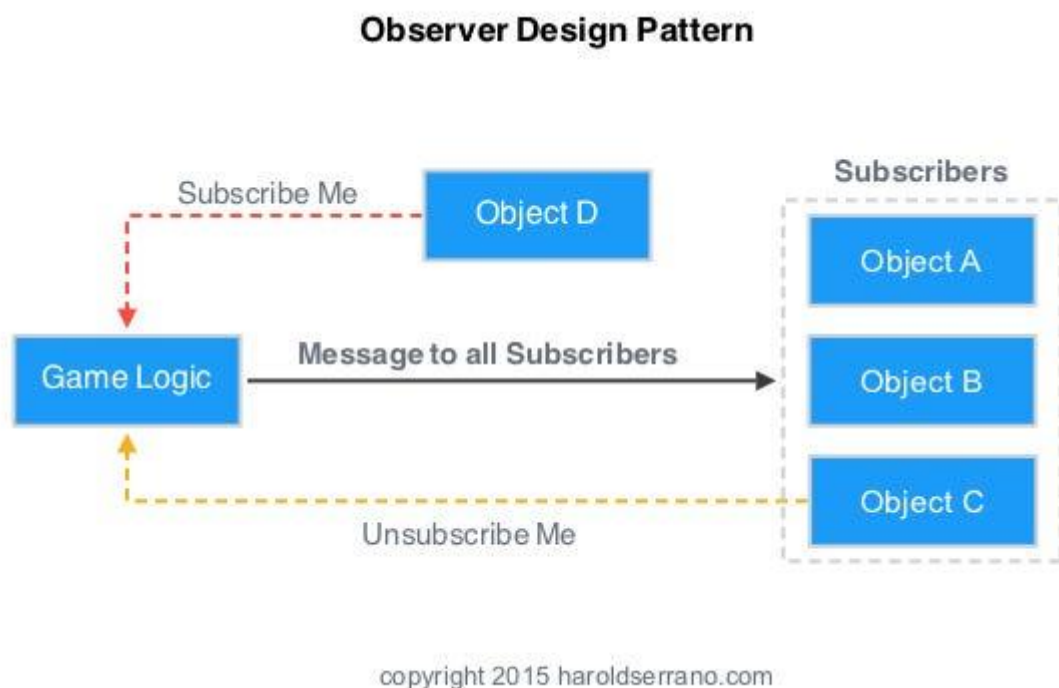


Hình 1.1: Hình so sánh Singleton với Normal Class

1.9.2. Observer Pattern

Observer Pattern định nghĩa mối quan hệ một-nhiều giữa các đối tượng, trong đó khi một đối tượng thay đổi trạng thái, tất cả các đối tượng phụ thuộc sẽ được thông báo tự động. Trong đề tài, pattern này được hiện thực qua lớp MessageManager với cơ chế đăng ký và phát sự kiện (event bus).

Cụ thể, các thành phần như bộ đếm thời gian, bộ điều khiển điểm số hay hệ thống UI đều đăng ký lắng nghe các sự kiện tương ứng thông qua interface IMessageHandle. Khi một sự kiện như OnScoreChanged hay OnTimerTick được phát ra, tất cả các subscriber sẽ nhận được thông báo mà không cần biết đến nhau, từ đó giảm sự phụ thuộc trực tiếp giữa các module.

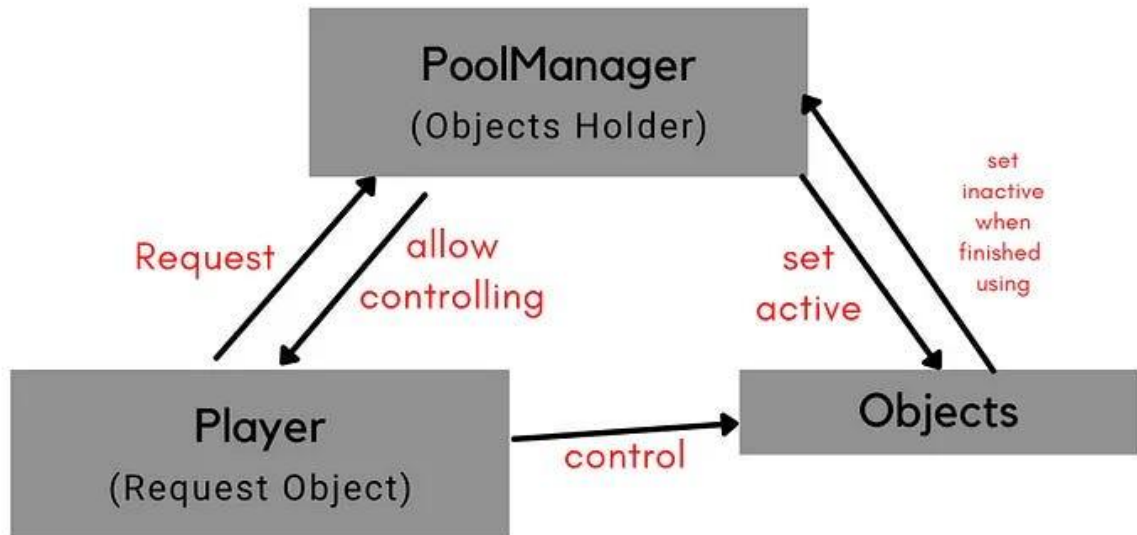


Hình 1.2: Hình của mẫu thiết kế Observer

1.9.3. Object Pooling Pattern

Object Pooling là kỹ thuật tối ưu hiệu năng bằng cách tái sử dụng các đối tượng đã được tạo ra thay vì liên tục khởi tạo và hủy chúng. Trong game, các thao tác Instantiate() và Destroy() tốn nhiều tài nguyên CPU và gây ra các đợt thu gom rác (Garbage Collection) làm giật lag. Đề tài sử dụng Object Pooling thông qua PoolManager và các lớp pool chuyên biệt như KitchenObjectPool cho nguyên liệu và món ăn, CounterPool cho các bàn bếp động trong Level Editor.

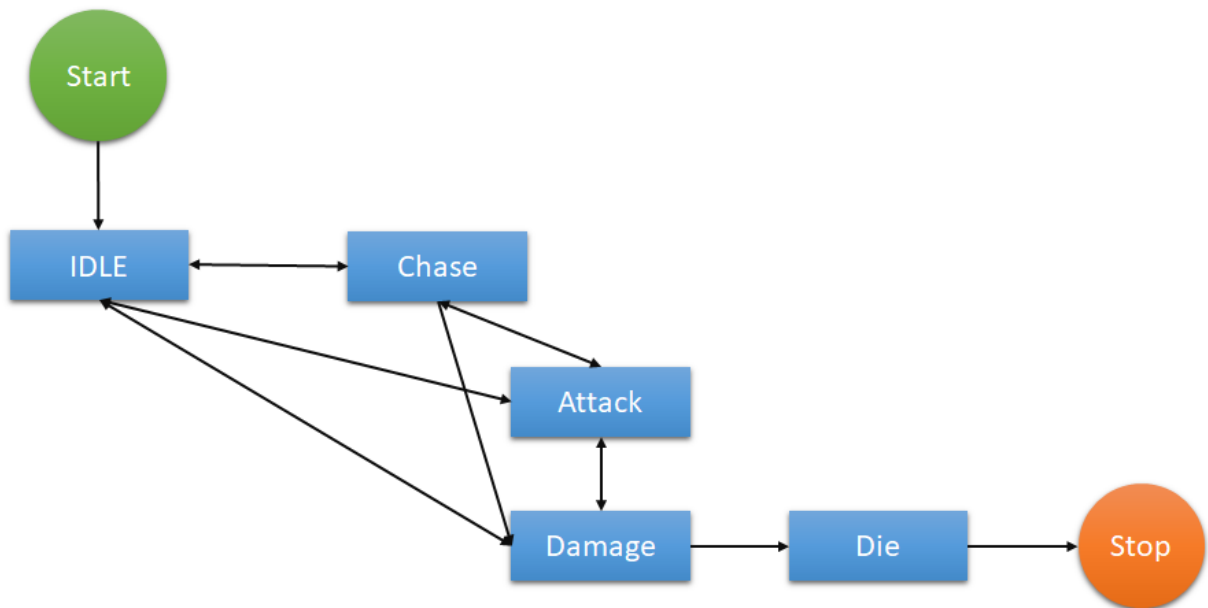
Object Pooling Design Pattern



Hình 1.3: Hình của mẫu thiết kế Object Pooling

1.9.4. Finite State Machine (FSM)

Máy trạng thái hữu hạn (Finite State Machine – FSM) là mô hình tính toán biểu diễn một hệ thống thông qua tập hợp các trạng thái xác định và các quy tắc chuyển đổi giữa chúng. FSM được áp dụng trong đề tài để quản lý vòng đời nấu nướng tại bếp lò (StoveCounter) với bốn trạng thái: Idle (trống), Frying (đang nấu), Fried (đã chín) và Burned (cháy khét). Việc chuyển đổi giữa các trạng thái được kiểm soát bởi hai bộ đếm thời gian fryingTimer và burningTimer, cập nhật sau mỗi tick mạng để đảm bảo đồng bộ giữa host và client.



Hình 1.4: Hình mẫu thiết kế Finite State Machine

1.10. Prefab trong Unity

Prefab là một cơ chế quan trọng trong Unity dùng để lưu trữ và tái sử dụng các đối tượng GameObject cùng toàn bộ thành phần (Component), thuộc tính và cấu hình của chúng. Prefab hoạt động như một khuôn mẫu (template), cho phép lập trình viên tạo nhiều bản sao của cùng một đối tượng mà vẫn đảm bảo tính nhất quán và dễ dàng quản lý. Trong đề tài, Prefab được sử dụng để xây dựng các thành phần của màn chơi như bàn bếp, quầy nguyên liệu, thùng rác và các đối tượng tương tác khác. Khi khởi tạo màn chơi, hệ thống sẽ tạo các đối tượng tương ứng từ Prefab đã được cấu hình sẵn, giúp giảm thời gian phát triển, tăng khả năng tái sử dụng mã nguồn và hỗ trợ mở rộng nội dung game một cách linh hoạt.

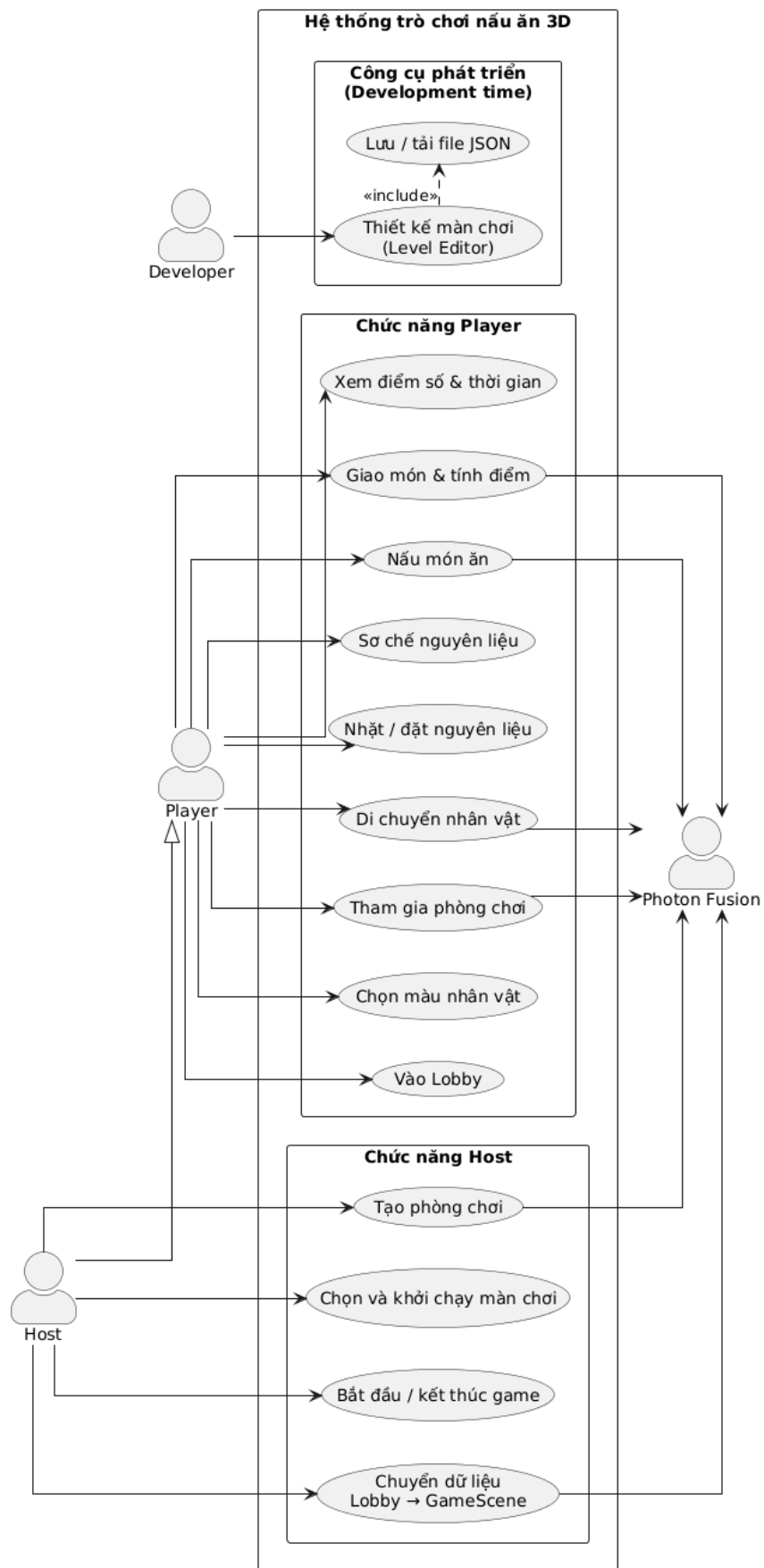
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích yêu cầu hệ thống

2.1.1. Các tác nhân (Actor) trong hệ thống

Hệ thống trò chơi nấu ăn 3D có ba tác nhân tương tác với hệ thống, gồm hai tác nhân người dùng và một tác nhân hệ thống:

- **Người chơi (Player):** tác nhân chính thực hiện các thao tác gameplay như di chuyển nhân vật, tương tác với nguyên liệu và dụng cụ bếp, hoàn thành yêu cầu nấu ăn và tham gia chế độ multiplayer trực tuyến.
- **Host:** người chơi đặc biệt kế thừa toàn bộ chức năng của Player, đồng thời có thêm các đặc quyền quản lý gồm: tạo phòng chơi, chọn màn chơi muốn khởi chạy, bắt đầu và kết thúc ván game, kiểm soát trạng thái toàn cục thông qua HasStateAuthority, và sử dụng hệ thống Level Editor để thiết kế bố cục bếp tùy chỉnh.
- **Photon Fusion Server:** tác nhân hệ thống, không tương tác trực tiếp với gameplay mà đóng vai trò chuyển tiếp dữ liệu mạng, đồng bộ các biến [Networked] từ Host về tất cả Client, và duy trì kết nối ổn định giữa các người chơi trong suốt ván game.
- **Developer:** tác nhân ngoài hệ thống runtime, sử dụng Level Editor trong môi trường Unity Editor để thiết kế bố cục nhà bếp và lưu lại dưới dạng các Prefab. Các file này được đóng gói vào thư mục Resources/Levels/ khi build game.



Hình 2.1: Sơ đồ ca sử dụng

2.1.2. Đặc tả use case chính

a. Tạo và tham gia phòng chơi

Trường	Nội dung
Mã use case	UC-01
Tên	Tạo và tham gia phòng chơi
Tác nhân	Host, Player, Photon Fusion Server
Mô tả	Host khởi tạo phòng chơi trực tuyến thông qua Photon Fusion. Player nhập tên phòng để tham gia. Sau khi các người chơi đã sẵn sàng hết, Host có thể chọn màn chơi và bắt đầu ván game.
Điều kiện đầu vào	Người dùng đang ở màn hình Lobby, có kết nối internet ổn định.
Luồng chính (Host)	1. Host nhập tên phòng và nhấn "Tạo phòng" 2. Hệ thống gọi <code>FusionNetworkRunner.StartGame()</code> với <code>GameMode.Host</code> 3. Photon Fusion tạo session và trả về kết quả thành công 4. Host chọn màu nhân vật tại Lobby 5. Host chọn màn chơi muốn khởi chạy 6. Host nhấn "Bắt đầu" → hệ thống chuyển scene sang <code>GameScene</code>
Luồng nhánh (Player tham gia)	1. Player chọn phòng và nhấn "Tham gia" 2. Hệ thống gọi <code>StartGame()</code> với <code>GameMode.Client</code> 3. Photon Fusion kết nối Player vào session của Host 4. Player chọn màu nhân vật và chờ Host bắt đầu
Luồng thay thế	Nếu phòng không tồn tại hoặc kết nối thất bại, hệ thống hiển thị thông báo lỗi và trở về màn hình Lobby.
Điều kiện kết thúc	Tất cả người chơi đã vào <code>GameScene</code> và nhân vật Player được spawn thành công.

Bảng 2.1: Bảng use case Tạo và tham gia phòng chơi

b. Sơ chế nguyên liệu

Trường	Nội dung
Mã use case	UC-02
Tên	Sơ chế nguyên liệu
Tác nhân	Player, Host (Photon Fusion)

Mô tả	Người chơi đặt nguyên liệu lên CuttingCounter và giữ phím tương tác để thực hiện thao tác cắt thái. Khi tiến trình đạt 100%, nguyên liệu chuyển thành dạng đã sơ chế.
Điều kiện đầu vào	Người chơi đang cầm nguyên liệu chưa sơ chế; nhân vật đứng trong vùng tương tác của CuttingCounter.
Luồng chính	1. Người chơi đặt nguyên liệu lên CuttingCounter 2. Người chơi giữ phím InteractAlternate 3. NetworkInputHandler thu thập input và gửi lên Host qua Photon Fusion 4. Host xác nhận và gọi CuttingCounter.InteractAlternate() 5. cuttingProgress tăng dần theo mỗi tick 6. MessageManager phát sự kiện OnCuttingProgress → UI cập nhật thanh tiến trình 7. Khi cuttingProgress đạt ngưỡng tối đa, KitchenObject được thay thế bằng nguyên liệu đã cắt
Luồng thay thế	Nếu người chơi thả phím trước khi hoàn thành, cuttingProgress giữ nguyên giá trị hiện tại và tiếp tục khi nhấn lại.
Điều kiện kết thúc	Nguyên liệu trên CuttingCounter chuyển thành dạng đã sơ chế, sẵn sàng để nấu hoặc kết hợp.

Bảng 2.2: Bảng use case Sơ chế nguyên liệu

c. Giao món ăn và tính điểm

Trường	Nội dung
Mã use case	UC-03
Tên	Giao món ăn và tính điểm
Tác nhân	Player, Host (Photon Fusion), GameplayManager
Mô tả	Người chơi mang đĩa đã hoàn thành đến DeliveryCounter. Hệ thống kiểm tra nội dung đĩa so với danh sách đơn hàng hiện tại và cộng điểm nếu đúng công thức.
Điều kiện đầu vào	Người chơi đang cầm đĩa chứa đầy đủ nguyên liệu; nhân vật đứng cạnh DeliveryCounter.
Luồng chính	1. Người chơi tương tác với DeliveryCounter 2. Host gọi DeliveryCounter.Interact() → ValidateRecipe(plate) 3. GameplayManager duyệt RecipeList để tìm công thức khớp 4. Phát sự kiện OnRecipeSuccess → cộng điểm 5. Phát tiếp OnScoreChanged → HUD cập nhật điểm số trên tất cả Client 6. Đĩa được despawn, đơn hàng tương ứng bị xóa khỏi danh sách

Luồng thay thế	Nếu không tìm thấy công thức khớp, hệ thống hiển thị thông báo giao hàng thất bại, không cộng điểm, đĩa giữ nguyên.
Điều kiện kết thúc	Điểm số được cập nhật và đồng bộ về tất cả Client thông qua biến [Networked] Score.

Bảng 2.3: Bảng use case Giao món ăn và tính điểm

d. Thiết kế và lưu màn chơi

Trường	Nội dung
Mã use case	UC-04
Tên	Thiết kế và lưu màn chơi
Tác nhân	Developer
Mô tả	Developer sử dụng hệ thống Level Editor trong môi trường Unity Editor để thiết kế bố cục nhà bếp bằng cách đặt các bàn bếp lên lưới tọa độ. Dữ liệu màn chơi được lưu dạng Prefab và đóng gói vào thư mục Resources/Levels/ khi build game.
Điều kiện đầu vào	Developer đang chạy project trong Unity Editor; scene LevelEditor đã được mở; danh sách Counter prefab khả dụng đã được load.
Luồng chính – Thiết kế và lưu	1. Developer mở scene Level Editor trong Unity Editor 2. Developer chọn loại Counter muốn đặt từ danh sách Inspector 3. Developer click vào ô trên lưới tọa độ để đặt Counter 4. LevelDesignerManager ghi nhận CounterData (loại Counter, vị trí Vector3, góc xoay Quaternion) vào danh sách placedCounters 5. Developer lặp lại bước 2–4 cho đến khi bố cục hoàn chỉnh 6. Developer nhập tên màn chơi và nhấn "Lưu" 7. Hệ thống lưu trữ các đối tượng dưới dạng Prefab, cho phép quản lý và tái sử dụng các thành phần của màn chơi một cách hiệu quả. 8. File Level_[Tên].prefab được ghi vào Assets/Resources/Levels/
Luồng nhánh - Chính sửa màn chơi đã có	1. Developer chọn file Prefab có sẵn từ danh sách 2. Hệ thống đọc file và deserialize thành LevelData 3. LevelDesignerManager Instantiate lại toàn bộ Counter theo dữ liệu đã lưu 4. Developer chỉnh sửa bố cục và lưu lại
Luồng thay thế	Nếu tên màn chơi đã tồn tại, hệ thống ghi đè file Prefab cũ. Nếu file Prefab bị lỗi định dạng khi đọc, hệ thống bỏ qua và ghi log cảnh báo ra Console.
Điều kiện kết thúc	File Level_[Tên].prefab được ghi thành công vào Resources/Levels/. File này sẽ được đóng gói cùng build game và có thể được Host chọn để khởi chạy tại runtime.

Bảng 2.4: Bảng use case Thiết kế và lưu màn chơi

e. Nấu món ăn

Trường	Nội dung
Mã use case	UC-05
Tên	Nấu món ăn trên bếp
Tác nhân	Người chơi, Host (Photon Fusion)
Điều kiện đầu vào	Người chơi đang cầm nguyên liệu đã sơ chế, nhân vật đứng cạnh StoveCounter
Luồng chính	1. Người chơi nhấn nút tương tác 2. NetworkInputHandler gửi input lên host 3. Host xác nhận và gọi StoveCounter.Interact() 4. FSM chuyển sang trạng thái Frying 5. FryingTimer tăng dần theo tick mạng 6. Khi đủ thời gian, FSM chuyển sang Fried 7. Người chơi lấy món ăn đã chín
Luồng thay thế	Nếu FryingTimer vượt quá BurningTimerMax, FSM chuyển sang Burned, hiển thị cảnh báo UI
Kết quả	Nguyên liệu được chuyển thành món ăn chín, sẵn sàng phục vụ

Bảng 2.5: Bảng use case Nấu món ăn

f. Chọn màu và chuyển scene sang GameScene

Trường	Nội dung
Mã use case	UC-06
Tên	Chuyển dữ liệu người chơi từ Lobby sang GameScene
Tác nhân	Host, Player, Photon Fusion Server
Mô tả	Khi Host bắt đầu ván game, hệ thống chuyển toàn bộ thông tin người chơi từ Lobby sang GameScene mà không mất dữ liệu trong quá trình chuyển scene.
Điều kiện đầu vào	Tất cả người chơi đã vào Lobby và chọn màu nhân vật. Host đã chọn màn chơi và nhấn "Bắt đầu".
Luồng chính (Host)	1. Host nhấn "Bắt đầu" → kích hoạt chuyển scene sang GameScene 2. Host đọc biến [Networked] PlayerColor từ tất cả LobbyPlayer còn tồn tại 3. Host Spawn prefab Player với màu sắc tương ứng 4. Hệ thống gán màu cho từng Player prefab vừa spawn 5. Host Despawn toàn bộ LobbyPlayer cũ 6. Kết nối mạng duy trì liên tục xuyên suốt quá trình

Luồng thay thế	Nếu một Client mất kết nối trong lúc chuyển scene, Host tiếp tục spawn các Player còn lại và bỏ qua Client đó.
Điều kiện kết thúc	Tất cả Player prefab được spawn đúng màu sắc trong GameScene. LobbyPlayer cũ đã được dọn sạch. Game bắt đầu.

Bảng 2.6: Bảng use case Chọn màn và chơi

g. Chọn và khởi chạy màn chơi

Trường	Nội dung
Mã use case	UC-07
Tên	Chọn và khởi chạy màn chơi
Tác nhân	Host
Mô tả	Sau khi tất cả người chơi đã vào Lobby, Host duyệt danh sách các màn chơi có sẵn (được load từ Resources/Levels/), chọn màn muốn chơi và khởi chạy ván game.
Điều kiện đầu vào	Host đang ở màn hình Lobby; đã có ít nhất một người chơi tham gia phòng; thư mục Resources/Levels/ có ít nhất một file Prefab hợp lệ.
Luồng chính	1. Hệ thống tự động load danh sách màn chơi bằng Resources.LoadAll() từ thư mục Resources/Levels/ 2. Danh sách tên màn chơi hiển thị trên UI Lobby 3. Host chọn màn chơi muốn khởi chạy 4. Hệ thống preview thông tin màn chơi (tên, số lượng Counter) 5. Host nhấn "Bắt đầu" 6. Hệ thống truyền thông tin màn chơi đã chọn qua biến [Networked] tới tất cả Client 7. Host kích hoạt chuyển scene sang GameScene 8. Trong GameScene, hệ thống sẽ tìm Prefab tương ứng và Instantiate toàn bộ Counter theo đúng bố cục đã lưu
Luồng thay thế	Nếu không có file Prefab nào trong Resources/Levels/, hệ thống load màn chơi mặc định được hardcode sẵn. Nếu file Prefab bị thiếu hoặc lỗi, hệ thống bỏ qua file đó và không hiển thị trong danh sách.
Điều kiện kết thúc	GameScene được load thành công với đúng bố cục búp theo màn chơi Host đã chọn. Tất cả Counter được Instantiate đúng vị trí, sẵn sàng để gameplay bắt đầu.

Bảng 2.7: Bảng use case Chọn và khởi chạy màn chơi

2.2. Kiến trúc của hệ thống

Hệ thống được thiết kế theo kiến trúc phân lớp nhằm tách biệt rõ ràng giữa giao diện người dùng, logic gameplay, xử lý mạng và quản lý dữ liệu. Việc phân chia này

giúp giảm sự phụ thuộc giữa các thành phần, tăng khả năng mở rộng và thuận lợi trong quá trình bảo trì.

Kiến trúc hệ thống bao gồm bốn lớp chính:

Lớp giao diện (UI Layer): Chịu trách nhiệm hiển thị thông tin tới người chơi như giao diện menu, giao diện phòng chờ, thời gian còn lại, điểm số, đơn hàng hiện tại và các thông báo trong trò chơi. Các thành phần giao diện được quản lý tập trung thông qua UIManager.

Lớp gameplay (Gameplay Layer): Đây là lớp quan trọng nhất của hệ thống, chịu trách nhiệm xử lý toàn bộ logic trò chơi. Các chức năng như điều khiển nhân vật, tương tác với bàn bếp, chế biến món ăn, tính điểm và quản lý tiến trình màn chơi đều được thực hiện tại lớp này.

Lớp mạng (Network Layer): Lớp mạng được xây dựng dựa trên Photon Fusion. Thành phần này chịu trách nhiệm đồng bộ dữ liệu giữa các người chơi, quản lý quyền điều khiển đối tượng, xử lý dữ liệu đầu vào từ người chơi và duy trì trạng thái nhất quán của trò chơi trên tất cả các máy khách.

Lớp dữ liệu (Data Layer): Lưu trữ và quản lý dữ liệu của hệ thống như thông tin công thức món ăn, dữ liệu màn chơi, dữ liệu người chơi và các cấu hình trò chơi. Các dữ liệu này được tổ chức dưới dạng Prefab hoặc ScriptableObject để thuận tiện cho việc mở rộng trong tương lai.

Việc phân tách hệ thống thành các lớp riêng biệt giúp giảm độ phức tạp của mã nguồn và tạo điều kiện thuận lợi cho việc phát triển các chức năng mới.

Sơ đồ lớp

2.3. Thiết kế kiến trúc hệ thống

2.3.1. Mô tả kiến trúc tổng thể

Hệ thống được tổ chức theo mô hình phân lớp, trong đó mỗi lớp chịu trách nhiệm một nhóm chức năng cụ thể và giao tiếp với nhau thông qua interface hoặc event bus. Kiến trúc tổng thể bao gồm bốn lớp chính:

- Lớp trình bày (Presentation Layer): quản lý giao diện người dùng, bao gồm UIManager, màn hình Lobby, HUD điểm số và thông báo.
- Lớp gameplay (Gameplay Layer): xử lý logic trò chơi, bao gồm PlayerController, hệ thống Counter và GameManager.
- Lớp mạng (Network Layer): quản lý kết nối và đồng bộ dữ liệu, bao gồm FusionNetworkRunner và NetworkInputHandler.
- Lớp dữ liệu (Data Layer): lưu trữ và quản lý cấu hình, bao gồm ScriptableObjects cho công thức nấu ăn, Level Editor và Prefab.

2.3.2. Thiết kế lớp Counter

Hệ thống Counter được thiết kế theo mô hình kế thừa với lớp cơ sở trừu tượng BaseCounter. Lớp này định nghĩa các thuộc tính và phương thức chung mà tất cả loại bàn bếp đều cần có, bao gồm điểm đặt đồ vật (counterTopPoint), phương thức Interact() và InteractAlternate(). Các lớp con kế thừa BaseCounter và override phương thức tương tác để triển khai hành vi riêng biệt: CuttingCounter tăng tiến trình cắt thái, StoveCounter quản lý FSM nấu nướng, DeliveryCounter kiểm tra công thức và cộng điểm.

Thiết kế này tuân theo nguyên tắc Open/Closed Principle trong SOLID: hệ thống mở để mở rộng thêm loại Counter mới nhưng đóng với việc thay đổi lớp BaseCounter, giúp dễ dàng bổ sung tính năng mới mà không ảnh hưởng đến code hiện tại.

2.4. Thiết kế cơ sở dữ liệu game

Thay vì sử dụng database quan hệ truyền thống, dữ liệu cấu hình của trò chơi được lưu trữ dưới hai hình thức: ScriptableObject Assets cho dữ liệu tĩnh như công thức nấu ăn và thông tin nguyên liệu, và file Prefab cho dữ liệu động như cấu hình màn chơi do người dùng tạo ra.

ScriptableObject là một loại asset đặc biệt trong Unity cho phép lưu trữ dữ liệu độc lập với scene và prefab. Ưu điểm chính là dữ liệu có thể chỉnh sửa trực tiếp trong Unity Editor mà không cần sửa code, phù hợp cho việc thêm công thức món ăn mới hay điều chỉnh thời gian nấu mà không ảnh hưởng đến mã nguồn.

2.5. Biểu đồ tuần tự

a. Biểu đồ tạo phòng chơi đến bắt đầu ván chơi

Biểu đồ tuần tự SD-01 mô tả toàn bộ quá trình khởi tạo một phiên chơi multiplayer, từ thời điểm Host tạo phòng cho đến khi màn chơi được load hoàn chỉnh và tất cả người chơi sẵn sàng bắt đầu. Luồng này liên quan đến 5 giai đoạn chính diễn ra tuần tự.

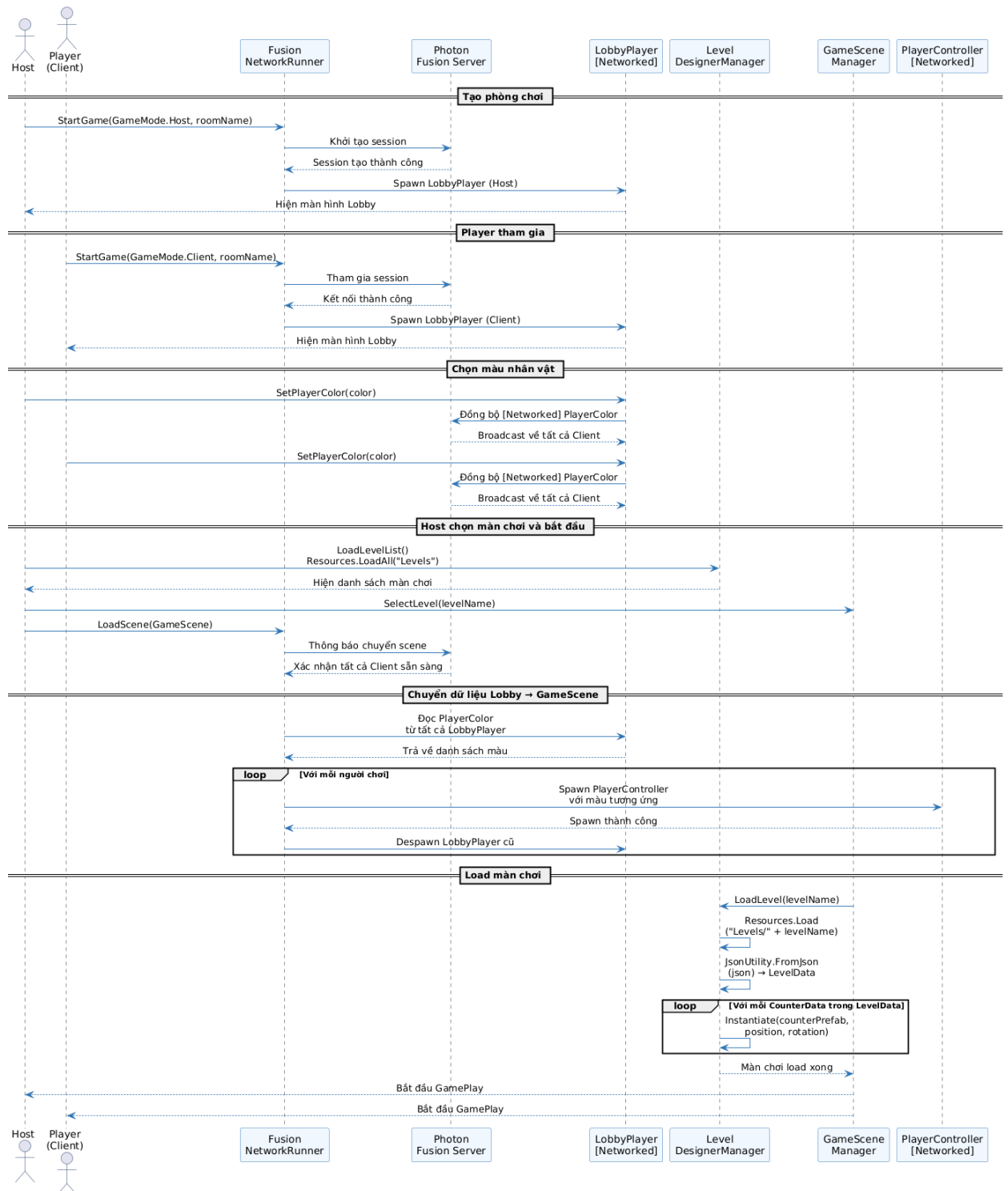
Giai đoạn 1 – Tạo phòng chơi: Host gọi `FusionNetworkRunner.StartGame()` với `GameMode.Host` và tên phòng tùy chọn. Photon Fusion Server tiếp nhận yêu cầu, khởi tạo session và trả về kết quả thành công. Sau đó hệ thống tự động Spawn đối tượng mạng `LobbyPlayer` cho Host và hiển thị màn hình Lobby.

Giai đoạn 2 – Player tham gia: Player nhập đúng tên phòng và gọi `StartGame()` với `GameMode.Client`. Photon Fusion Server xác nhận kết nối và Spawn thêm một `LobbyPlayer` cho Client. Lúc này cả Host và Player đều đang ở màn hình Lobby với đối tượng `LobbyPlayer` riêng của mình.

Giai đoạn 3 – Chọn màu nhân vật: Mỗi người chơi chọn màu nhân vật mong muốn. Thông tin màu sắc được lưu vào biến `[Networked] PlayerColor` trên `LobbyPlayer` tương ứng. Photon Fusion tự động broadcast giá trị này về tất cả Client, đảm bảo mọi người đều thấy màu sắc của nhau trong Lobby.

Giai đoạn 4 – Chuyển scene và truyền dữ liệu: Host chọn màn chơi từ danh sách được load qua `Resources.LoadAll("Levels")` rồi nhấn "Bắt đầu". Đây là giai đoạn kỹ thuật phức tạp nhất: trước khi chuyển scene, hệ thống đọc `PlayerColor` từ tất cả `LobbyPlayer` còn tồn tại, sau đó lần lượt Spawn prefab `PlayerController` với đúng màu sắc tương ứng cho từng người chơi, rồi mới Despawn toàn bộ `LobbyPlayer` cũ. Cơ chế này đảm bảo không mất dữ liệu người chơi trong quá trình chuyển scene trong khi kết nối mạng vẫn duy trì liên tục.

Giai đoạn 5 – Load màn chơi: `LevelDesignerManager` tải các Prefab tương ứng với màn chơi đã chọn từ thư mục `Resources` thông qua `Resources.Load()`. Sau đó, hệ thống khởi tạo các Prefab bằng phương thức `Instantiate()` để tạo các đối tượng trong màn chơi với cấu hình đã được thiết lập sẵn. Khi toàn bộ đối tượng được tạo hoàn tất, `GameSceneManager` thông báo cho tất cả người chơi rằng màn chơi đã sẵn sàng và gameplay bắt đầu.



Hình 2.3: Sơ đồ tuần tự tạo phòng chơi và bắt đầu ván chơi

b. Biểu đồ nấu và giao món ăn

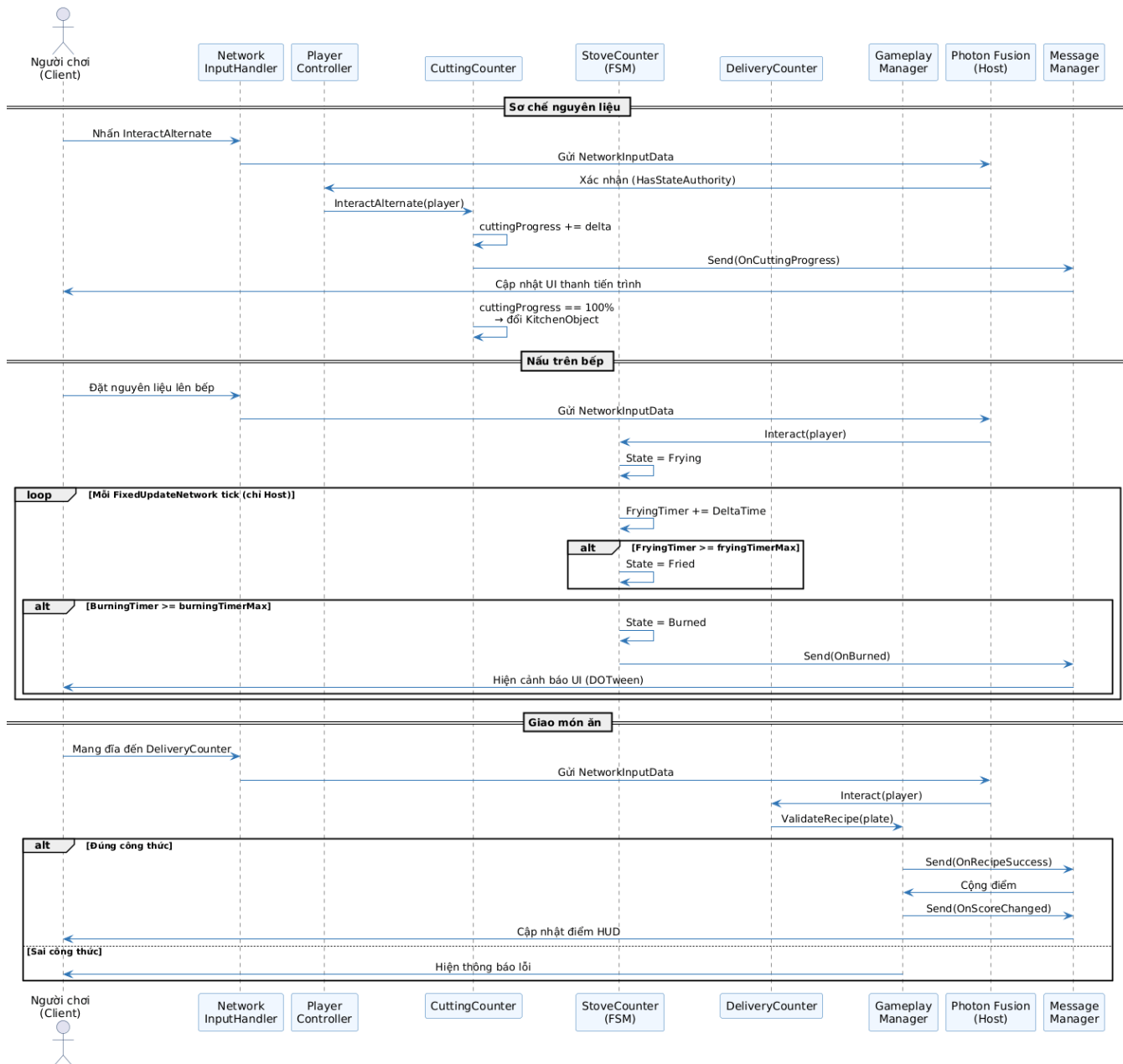
Biểu đồ tuần tự SD-02 mô tả vòng lặp gameplay cốt lõi của trò chơi, bao gồm ba giai đoạn nối tiếp nhau: sơ chế nguyên liệu, nấu món ăn và giao món để tính điểm. Biểu đồ này thể hiện đồng thời ba kỹ thuật quan trọng nhất của hệ thống: cơ chế đồng bộ

input qua Photon Fusion, Finite State Machine tại StoveCounter và Observer Pattern qua MessageManager.

Giai đoạn 1 – Sơ chế nguyên liệu: Người chơi giữ phím `InteractAlternate` tại `CuttingCounter`. `NetworkInputHandler` thu thập trạng thái phím bấm, đóng gói vào struct `NetworkInputData` và gửi lên Host qua callback `OnInput` của Photon Fusion. Host xác nhận input thông qua `HasStateAuthority` và gọi `CuttingCounter.InteractAlternate()`. Biến `cuttingProgress` tăng dần theo mỗi network tick. Sau mỗi lần tăng, `CuttingCounter` phát sự kiện `OnCuttingProgress` qua `MessageManager` để UI cập nhật thanh tiến trình. Khi `cuttingProgress` đạt ngưỡng tối đa, `KitchenObject` trên bàn được thay thế bằng nguyên liệu đã sơ chế.

Giai đoạn 2 – Nấu trên bếp: Người chơi đặt nguyên liệu lên `StoveCounter`, kích hoạt FSM chuyển từ trạng thái `Idle` sang `Frying`. Từ thời điểm này, mỗi `FixedUpdateNetwork` tick — chỉ chạy trên Host do kiểm tra `HasStateAuthority` — hệ thống tăng `FryingTimer`. Khi `FryingTimer` đạt `fryingTimerMax`, FSM chuyển sang trạng thái `Fried` và người chơi có thể lấy món ra. Nếu người chơi không lấy kịp thời, `BurningTimer` bắt đầu tăng. Khi vượt `burningTimerMax`, FSM chuyển sang `Burned`, `MessageManager` phát sự kiện `OnBurned` kích hoạt animation cảnh báo nhấp nháy qua DOTween trên tất cả Client. Toàn bộ trạng thái FSM được lưu trong biến `[Networked] State`, đảm bảo nhất quán trên mọi máy.

Giai đoạn 3 – Giao món và tính điểm: Người chơi mang đĩa đến `DeliveryCounter` và tương tác. Host gọi `ValidateRecipe(plate)` trên `GameplayManager`, hệ thống duyệt danh sách `RecipeList` hiện tại để tìm công thức khớp với nội dung đĩa. Nếu tìm thấy, `GameplayManager` phát lần lượt hai sự kiện qua `MessageManager`: `OnRecipeSuccess` để xử lý logic cộng điểm, sau đó `OnScoreChanged` để tất cả subscriber cập nhật giao diện. Điểm số được lưu trong biến `[Networked] Score` và đồng bộ về tất cả Client ngay lập tức. Nếu công thức không khớp, hệ thống chỉ hiển thị thông báo lỗi mà không thay đổi điểm số hay trạng thái đĩa.



Hình 2.4: Sơ đồ tuần tự nấu và giao món ăn

2.6. Biểu đồ hành động

a. Luồng gameplay chính

Mô tả toàn bộ vòng đời của một ván game, từ thời điểm người chơi vào Lobby cho đến khi màn hình kết quả được hiển thị. Biểu đồ sử dụng mô hình swimlane với ba làn song song gồm Player (Client), Host và Photon Fusion (Server) nhằm thể hiện rõ trách nhiệm của từng thành phần trong hệ thống.

Giai đoạn khởi tạo: Luồng bắt đầu từ làn Player với các bước vào Lobby và chọn màu nhân vật. Song song đó, Host thực hiện các thao tác quản lý phòng gồm tạo phòng với GameMode.Host, chọn màn chơi từ danh sách và nhấn Bắt đầu. Photon Fusion

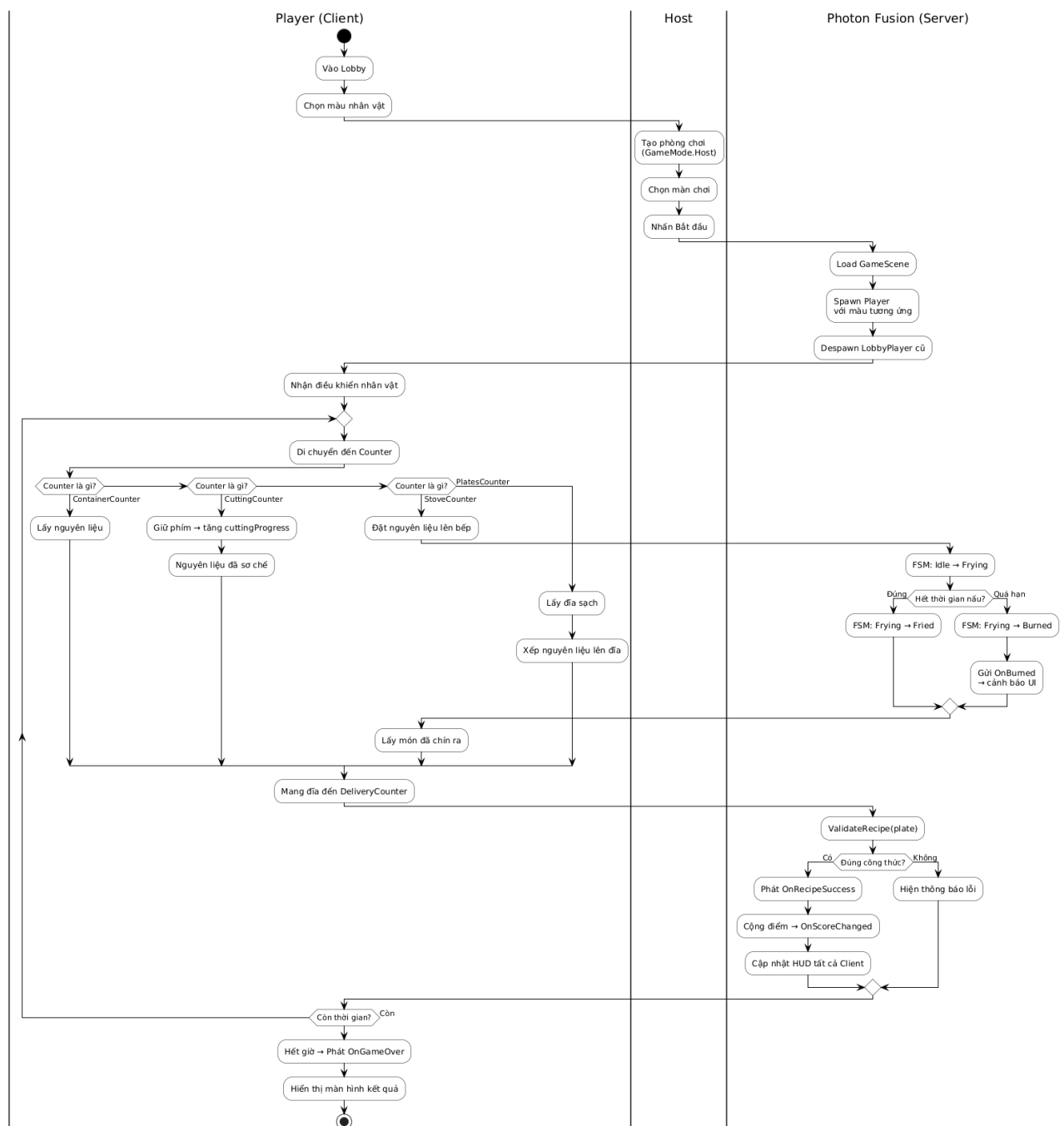
Server tiếp nhận lệnh từ Host, load GameScene, Spawn PlayerController với đúng màu sắc tương ứng đã lưu trong LobbyPlayer, sau đó Despawn toàn bộ LobbyPlayer cũ để dọn sạch dữ liệu Lobby.

Giai đoạn gameplay (vòng lặp): Sau khi nhận được quyền điều khiển nhân vật, Player bước vào vòng lặp gameplay chính được biểu diễn bằng cấu trúc repeat...while. Mỗi vòng lặp bắt đầu bằng thao tác di chuyển đến Counter, sau đó rẽ nhánh theo loại Counter tương tác:

- **ContainerCounter:** Player lấy nguyên liệu thô từ hộp chứa, không có điều kiện phụ.
- **CuttingCounter:** Player giữ phím tương tác để tăng dần cuttingProgress cho đến khi nguyên liệu chuyển thành dạng đã sơ chế.
- **StoveCounter:** Luồng chuyển sang làn Photon Fusion Server để xử lý FSM. Hệ thống chuyển trạng thái từ Idle sang Frying và bắt đầu đếm thời gian. Nếu FryingTimer đạt ngưỡng đúng hạn, FSM chuyển sang Fried và Player lấy món ra. Nếu quá hạn, FSM chuyển sang Burned, MessageManager phát sự kiện OnBurned kích hoạt cảnh báo UI nhấp nháy qua DOTween.
- **PlatesCounter:** Player lấy đĩa sạch và xếp nguyên liệu đã chế biến lên đĩa.

Sau khi hoàn thành quy trình tại Counter, Player mang đĩa đến DeliveryCounter. Luồng chuyển sang làn Photon Fusion Server để thực hiện ValidateRecipe(plate). Nếu công thức đúng, hệ thống lần lượt phát OnRecipeSuccess để cộng điểm và OnScoreChanged để cập nhật HUD trên tất cả Client. Nếu sai, chỉ hiển thị thông báo lỗi mà không thay đổi điểm số.

Điều kiện kết thúc vòng lặp: Vòng lặp tiếp tục cho đến khi GameTimer về 0. Khi hết giờ, hệ thống phát sự kiện OnGameOver qua MessageManager, tất cả Client nhận được thông báo và chuyển sang màn hình kết quả hiển thị điểm số cuối cùng.



Hình 2.5: Sơ đồ hành động cho gameplay chính

CHƯƠNG 3: CÀI ĐẶT, KIỂM THỬ VÀ ĐÁNH GIÁ

3.1. Tổng quan quá trình triển khai

Hệ thống trò chơi được triển khai trên nền tảng Unity bằng ngôn ngữ lập trình C#. Quá trình phát triển áp dụng phương pháp lập trình hướng đối tượng nhằm tổ chức mã nguồn rõ ràng và thuận lợi cho việc mở rộng sau này. Các chức năng chính của trò chơi được chia thành nhiều hệ thống riêng biệt như quản lý gameplay, điều khiển nhân vật, quản lý nguyên liệu và món ăn, quản lý bản đồ bếp và hệ thống hiển thị.

Trong trò chơi nấu ăn, người chơi sẽ điều khiển nhân vật di chuyển trong khu vực nhà bếp để thực hiện các thao tác như thu thập nguyên liệu, chế biến món ăn, sử dụng các dụng cụ bếp và phục vụ món ăn cho khách hàng. Vì vậy, hệ thống gameplay được thiết kế để xử lý các tương tác giữa nhân vật với môi trường bếp một cách linh hoạt và trực quan.

Các thành phần chính của hệ thống bao gồm hệ thống nhận input từ người chơi, hệ thống điều khiển nhân vật, hệ thống quản lý nguyên liệu và công thức món ăn, cùng với hệ thống quản lý tiến trình của màn chơi. Việc tách các chức năng thành những module riêng giúp mã nguồn dễ quản lý, giảm sự phụ thuộc giữa các thành phần và tạo điều kiện thuận lợi cho việc bổ sung thêm món ăn, nguyên liệu hoặc cơ chế gameplay mới trong tương lai.

3.2. Triển khai quản lý trò chơi (gameplay) và xử lý đầu vào (input)

3.2.1. Mục tiêu thiết kế

Trong trò chơi nấu ăn, người chơi cần thực hiện nhiều thao tác khác nhau như di chuyển nhân vật, nhặt nguyên liệu, chế biến món ăn và phục vụ khách hàng trong thời gian giới hạn. Vì vậy, hệ thống quản lý gameplay và input cần được thiết kế rõ ràng để đảm bảo quá trình chơi diễn ra mượt mà và hạn chế lỗi trong quá trình tương tác.

Hệ thống quản lý gameplay trong đồ án này được xây dựng nhằm đạt được các mục tiêu sau:

- Tiếp nhận và xử lý chính xác thao tác điều khiển của người chơi
- Đồng bộ giữa điều khiển nhân vật, animation và trạng thái gameplay
- Quản lý tiến trình của màn chơi như thời gian, nhiệm vụ và điểm số
- Đảm bảo các thao tác tương tác với môi trường bếp được xử lý đúng thứ tự
- Tạo nền tảng dễ mở rộng để bổ sung thêm món ăn, nguyên liệu và cơ chế gameplay mới

3.2.2. Kiến trúc tổng thể

Hệ thống quản lý gameplay được tổ chức thành các thành phần chính sau:

- **InputHandler**: tiếp nhận thao tác điều khiển từ người chơi
- **GameplayManager**: quản lý trạng thái chung của trò chơi
- **PlayerController**: điều khiển di chuyển và hành động của nhân vật
- **KitchenManager**: quản lý các đối tượng trong khu vực bếp

Cách tổ chức này giúp tách biệt rõ ràng giữa việc nhận input, xử lý gameplay và điều khiển nhân vật, từ đó giúp hệ thống dễ hiểu và dễ bảo trì.

3.2.3. Xử lý input người chơi

Việc tiếp nhận thao tác điều khiển được thực hiện thông qua lớp **InputHandler**. Lớp này có nhiệm vụ lắng nghe các thao tác của người chơi như:

- Di chuyển nhân vật
- Nhặt hoặc đặt nguyên liệu
- Tương tác với các dụng cụ bếp
- Thực hiện các hành động chế biến món ăn

Sau khi nhận được input, hệ thống sẽ chuyển thông tin này tới các thành phần liên quan như **PlayerController** để thực hiện hành động tương ứng. Cách tiếp cận này giúp tách biệt hoàn toàn việc xử lý input khỏi logic gameplay, từ đó hạn chế lỗi và tăng khả năng mở rộng hệ thống.

3.3. Triển khai logic di chuyển

3.3.1. Vai trò của lớp GameLogic

Logic gameplay chịu trách nhiệm xử lý toàn bộ các hoạt động diễn ra trong trò chơi, bao gồm tương tác giữa nhân vật với nguyên liệu, dụng cụ bếp và hệ thống nhiệm vụ của màn chơi.

Khi người chơi thực hiện một hành động, hệ thống gameplay sẽ xử lý các bước sau:

- Xác định loại hành động của người chơi
- Kiểm tra điều kiện tương tác với đối tượng trong môi trường
- Thực hiện các thao tác chế biến món ăn
- Cập nhật trạng thái của nguyên liệu hoặc món ăn
- Cập nhật điểm số và tiến trình màn chơi

Việc tách riêng logic gameplay giúp đảm bảo rằng mọi hành vi trong trò chơi được xử lý một cách thống nhất.

3.3.2. Luồng xử lý hành động của trò chơi

Khi người chơi thực hiện một thao tác tương tác, hệ thống gameplay sẽ xử lý theo trình tự sau:

- Nhận input từ người chơi

- Xác định đối tượng mà nhân vật đang tương tác
- Kiểm tra trạng thái của đối tượng
- Thực hiện hành động tương ứng (nhặt, đặt, chế biến hoặc phục vụ món ăn)
- Cập nhật trạng thái của hệ thống gameplay

Quy trình này giúp đảm bảo rằng các thao tác trong trò chơi được xử lý một cách logic và nhất quán.

3.3.3. Hệ thống chế biến món ăn

Một trong những cơ chế gameplay quan trọng của trò chơi là hệ thống chế biến món ăn. Mỗi món ăn được tạo thành từ các nguyên liệu cụ thể và cần trải qua một số bước chế biến như:

- Chuẩn bị nguyên liệu
- Sơ chế nguyên liệu
- Nấu hoặc kết hợp các nguyên liệu
- Hoàn thành món ăn

Hệ thống này cho phép trò chơi dễ dàng bổ sung thêm các món ăn mới bằng cách thay đổi dữ liệu cấu hình.

3.4. Triển khai hệ thống bản đồ của mỗi cấp độ

Hệ thống bản đồ trong trò chơi đại diện cho không gian nhà bếp nơi người chơi thực hiện các thao tác nấu ăn. Bản đồ bao gồm nhiều khu vực chức năng khác nhau như:

- Khu vực chứa nguyên liệu
- Khu vực sơ chế
- Khu vực nấu ăn
- Khu vực phục vụ món ăn

Mỗi khu vực được thiết kế dưới dạng các đối tượng trong Unity, cho phép nhân vật tương tác trực tiếp. Việc tổ chức bản đồ theo các khu vực chức năng giúp người chơi dễ dàng thực hiện các thao tác trong quá trình nấu ăn.

Ngoài ra, hệ thống bản đồ cũng cho phép thiết kế nhiều màn chơi với bố cục nhà bếp khác nhau, từ đó tạo ra các thử thách đa dạng cho người chơi.

3.5. Triển khai các thực thể trong trò chơi

3.5.1. Tổng quan thiết kế thực thể

Trong trò chơi, các thực thể là những đối tượng trực tiếp tham gia vào gameplay như nhân vật, nguyên liệu, món ăn và các dụng cụ bếp. Để đảm bảo tính linh hoạt và dễ mở rộng, các thực thể được thiết kế theo hướng tách biệt giữa dữ liệu và logic xử lý.

Các thông tin đặc trưng của từng loại đối tượng được lưu trữ dưới dạng dữ liệu cấu hình, trong khi hành vi của chúng được triển khai thông qua các script trong Unity.

3.5.2. Nhân vật của người chơi

Nhân vật người chơi là đối tượng trung tâm của trò chơi và được điều khiển trực tiếp thông qua input. Nhân vật có khả năng:

- Di chuyển trong khu vực nhà bếp
- Nhặt và mang nguyên liệu
- Tương tác với các dụng cụ bếp
- Chế biến và phục vụ món ăn

Hệ thống điều khiển nhân vật được thiết kế để đảm bảo việc di chuyển và tương tác diễn ra mượt mà.

3.5.3. Nguyên liệu và món ăn

Nguyên liệu là các đối tượng mà người chơi có thể thu thập và sử dụng để chế biến món ăn. Mỗi nguyên liệu có các thuộc tính riêng như:

- Tên nguyên liệu
- Hình ảnh hiển thị
- Khả năng sử dụng trong các công thức món ăn

Khi các nguyên liệu được kết hợp theo đúng công thức, hệ thống sẽ tạo ra món ăn hoàn chỉnh để người chơi phục vụ cho khách hàng.

3.5.4. Dụng cụ bếp

Các dụng cụ bếp là những đối tượng hỗ trợ quá trình chế biến món ăn như bàn chế biến, bếp nấu hoặc khu vực phục vụ món ăn. Mỗi dụng cụ có chức năng riêng và cho phép người chơi thực hiện các thao tác khác nhau trong quá trình nấu ăn.

Việc thiết kế các dụng cụ bếp dưới dạng các đối tượng riêng biệt giúp hệ thống dễ dàng mở rộng khi bổ sung thêm cơ chế gameplay mới.

3.6. Triển khai hiển thị và tài nguyên

Trong quá trình phát triển trò chơi, hệ thống hiển thị và quản lý tài nguyên đóng vai trò quan trọng trong việc tạo ra trải nghiệm trực quan cho người chơi. Trong đề án này, hệ thống hiển thị được triển khai thông qua Unity với các thành phần như mô hình 3D, vật liệu, ánh sáng và hệ thống camera nhằm xây dựng môi trường nhà bếp sinh động.

Các đối tượng trong trò chơi như nhân vật, nguyên liệu, dụng cụ bếp và món ăn được xây dựng dưới dạng các mô hình 3D và được gắn vào các **GameObject** trong Unity. Mỗi đối tượng được kết hợp với các thành phần hiển thị như **Mesh Renderer** và **Material** để thể hiện hình dạng và màu sắc trong không gian trò chơi. Ngoài ra, các

animation đơn giản cũng được sử dụng để thể hiện các hành động của nhân vật như di chuyển, nhặt nguyên liệu hoặc thực hiện thao tác chế biến món ăn.

Hệ thống camera được thiết kế theo góc nhìn cố định từ trên xuống nhằm giúp người chơi dễ dàng quan sát toàn bộ khu vực nhà bếp và thực hiện các thao tác điều khiển một cách thuận tiện. Cách bố trí này phù hợp với lối chơi phối hợp và quản lý nhiều hoạt động diễn ra đồng thời trong môi trường bếp.

Bên cạnh đó, các tài nguyên của trò chơi như mô hình, hình ảnh, âm thanh và hiệu ứng được tổ chức và quản lý thông qua hệ thống tài nguyên của Unity. Việc phân loại tài nguyên theo từng nhóm như nhân vật, nguyên liệu, môi trường và giao diện giúp quá trình phát triển và bảo trì dự án trở nên thuận tiện hơn. Ngoài ra, việc tái sử dụng các tài nguyên chung cũng giúp giảm dung lượng trò chơi và tối ưu hiệu năng khi chạy.

Nhờ cách tổ chức hợp lý giữa hệ thống hiển thị và tài nguyên, trò chơi có thể đảm bảo được tính trực quan, dễ quan sát và tạo cảm giác sinh động cho môi trường nấu ăn trong quá trình chơi.

3.7. Triển khai hệ thống mạng Multiplayer với Photon Fusion

3.7.1. Khởi tạo kết nối và quản lý phòng chơi

Hệ thống mạng được khởi tạo thông qua lớp FusionNetworkRunner, kế thừa Singleton để đảm bảo chỉ tồn tại một NetworkRunner duy nhất trong toàn bộ vòng đời game. Khi người chơi chọn tạo phòng, hệ thống sẽ gọi phương thức StartGame() với GameMode.Host, còn khi tham gia phòng sẽ dùng GameMode.Client. Sau khi kết nối thành công, Photon Fusion tự động đồng bộ các đối tượng có thuộc tính [Networked] giữa tất cả người chơi.

Đoạn code minh họa khởi tạo kết nối:

```
public async void StartHost(string roomName)
{
    var result = await _runner.StartGame(new StartGameArgs()
    {
        GameMode = GameMode.Host,
        SessionName = roomName,
        Scene = SceneRef.FromIndex(gameSceneIndex),
        SceneManager = SceneManager.Default,
    });
    gameObject.AddComponent<NetworkSceneManagerDefault>();
    if (!result.Ok) Debug.LogError(result.ShutdownReason);
}
```

3.7.2. Đồng bộ trạng thái nhân vật và input người chơi

Dữ liệu đầu vào của người chơi được thu thập tại mỗi client thông qua lớp `NetworkInputHandler`, đóng gói vào struct `NetworkInputData` chứa hướng di chuyển (`MoveDirection`) và các trạng thái nút bấm (`IsInteracting`, `IsInteractingAlternate`). Struct này được gửi lên host thông qua callback `OnInput` của `INetworkRunnerCallbacks`, đảm bảo tất cả input đều đi qua một đường dẫn duy nhất và được xử lý theo thứ tự tick.

Đoạn code minh họa thu thập input:

```
public void OnInput(NetworkRunner runner, NetworkInput input)
{
    var data = new NetworkInputData();
    data.MoveDirection = new Vector2(
        Input.GetAxisRaw("Horizontal"),
        Input.GetAxisRaw("Vertical")
    );
    data.IsInteracting = Input.GetKey(KeyCode.E);
    data.IsInteractingAlternate = Input.GetKey(KeyCode.F);
    input.Set(data);
}
```

3.7.3. Quản lý quyền kiểm soát trạng thái (State Authority)

Photon Fusion phân quyền kiểm soát trạng thái thông qua thuộc tính `HasStateAuthority`. Chỉ host mới có quyền thay đổi các biến `[Networked]`, còn client chỉ đọc giá trị đó để hiển thị. Cơ chế này được áp dụng triệt để trong lớp `StoveCounter`: chỉ khi `HasStateAuthority` trả về true, host mới cập nhật `FryingTimer` và chuyển đổi `StoveState`. Điều này đảm bảo toàn bộ trạng thái nấu ăn là nhất quán trên tất cả máy.

Đoạn code minh họa FSM đồng bộ mạng:

```
public override void FixedUpdateNetwork()
{
    if (!HasStateAuthority) return;
    if (State == StoveState.Frying)
    {
        FryingTimer += Runner.DeltaTime;
        if (FryingTimer >= currentFryingRecipeSO.fryingTimerMax)
        {
            State = StoveState.Fried;
            FryingTimer = 0f;
        }
    }
}
```

```
}  
}
```

3.7.4. Chuyển dữ liệu từ Lobby sang GameScene

Khi người chơi ở Lobby, thông tin cá nhân như màu sắc nhân vật được lưu trên đối tượng mạng LobbyPlayer với các biến [Networked]. Khi host kích hoạt chuyển scene sang GameScene, hệ thống sẽ đọc dữ liệu màu từ tất cả LobbyPlayer còn tồn tại, spawn prefab Player tương ứng và gán màu phù hợp, sau đó despawn LobbyPlayer cũ. Quy trình này đảm bảo không mất dữ liệu người chơi trong quá trình chuyển scene dù kết nối mạng vẫn duy trì liên tục.

3.8. Triển khai hệ thống Level Editor

3.8.1. Kiến trúc tổng thể của Level Editor

Hệ thống Level Editor cho phép người chơi tự thiết kế bố cục nhà bếp bằng cách đặt các bàn bếp vào vị trí mong muốn trên lưới tọa độ (grid). Lớp LevelDesignerManager đóng vai trò điều phối toàn bộ quá trình: nhận input kéo thả từ người dùng, quản lý danh sách các Counter đã đặt và điều phối việc lưu/tải dữ liệu màn chơi.

3.8.2. Tuần tự hóa và lưu trữ dữ liệu

Dữ liệu màn chơi được lưu trữ dưới dạng Prefab trong thư mục Resources/Levels. Khi người dùng nhấn lưu, hệ thống thu thập toàn bộ các đối tượng trong màn chơi cùng các thông tin cấu hình cần thiết như vị trí camera, danh sách bàn bếp và nguyên liệu, sau đó sử dụng PrefabUtility.SaveAsPrefabAssetAndConnect() để tạo hoặc cập nhật Prefab tương ứng. Khi tải màn chơi, hệ thống sử dụng AssetDatabase.LoadAssetAtPath() để nạp Prefab đã lưu và khởi tạo nó vào Scene bằng PrefabUtility.InstantiatePrefab(). Đồng thời, các thông tin bổ sung được lưu trong thành phần LevelPrefabData cũng được khôi phục để tái tạo chính xác trạng thái của màn chơi.

Đoạn code minh họa lưu màn chơi:

```
public void SaveLevel(string levelName)
{
    LevelPrefabData prefabData =
counterParent.GetComponent<LevelPrefabData>();
    if (prefabData == null) prefabData =
counterParent.gameObject.AddComponent<LevelPrefabData>();
    if (levelPreviewCamera != null)
    {
        prefabData.cameraPosition = levelPreviewCamera.transform.position;
        prefabData.cameraEulerAngles =
```

```

levelPreviewCamera.transform.eulerAngles;
    }
    prefabData.baseCounters.Clear();
    prefabData.baseCounters.AddRange(counterParent.GetComponentsInChildren
<BaseCounter>(true));

    prefabData.kitchenObjects.Clear();
    prefabData.kitchenObjects.AddRange(counterParent.GetComponentsInChildren<KitchenObject>(true));
    counterParent.name = "Level_" + levelName;
    string folderPath = "Assets/Resources/Levels";
    if (!System.IO.Directory.Exists(folderPath))
        System.IO.Directory.CreateDirectory(folderPath);
    string prefabPath = folderPath + "/" + levelName + ".prefab";
    UnityEditor.PrefabUtility.SaveAsPrefabAssetAndConnect(
        counterParent.gameObject, prefabPath,
        UnityEditor.InteractionMode.UserAction, out bool success);
    if (success)
        Debug.Log($"[LevelDesigner] Prefab saved: {prefabPath}");
    else
        Debug.LogError($"[LevelDesigner] Failed to save prefab: {prefabPath}");

    UnityEditor.AssetDatabase.Refresh();
    Debug.LogWarning($"[LevelDesigner] SaveLevel is only available in the Unity
Editor.");
}

```

3.9. Kết quả đạt được

Sau quá trình nghiên cứu và phát triển, đề tài đã xây dựng thành công trò chơi nấu ăn 3D hỗ trợ cả chế độ chơi đơn và nhiều người chơi thông qua Photon Fusion. Các chức năng chính của hệ thống đều được triển khai hoàn chỉnh và hoạt động ổn định.

Các chức năng đã hoàn thành:

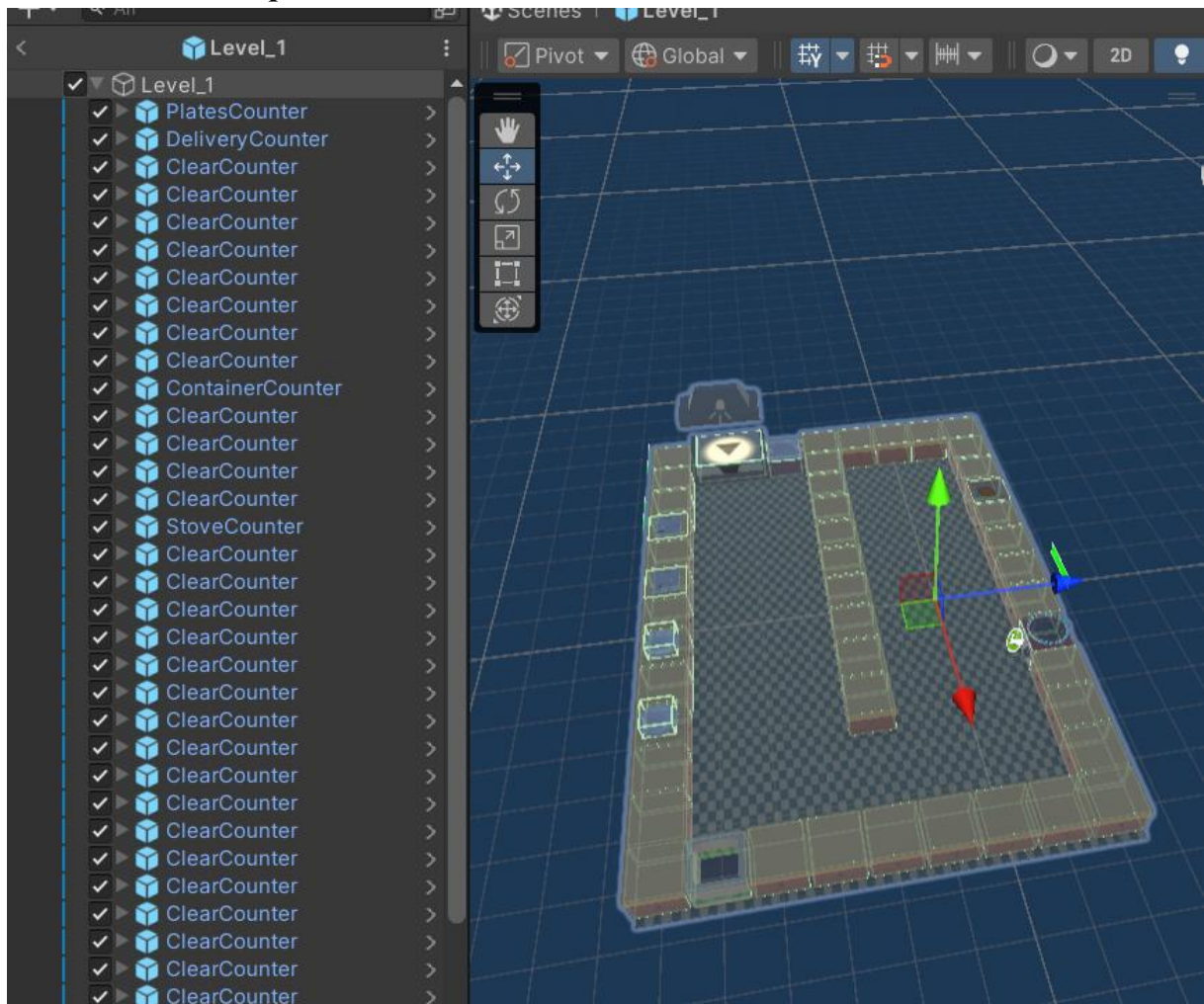
- Điều khiển nhân vật trong môi trường 3D.
- Hệ thống tương tác với nguyên liệu và bàn bếp.
- Hệ thống chế biến món ăn theo công thức.
- Hệ thống tính điểm và giao món ăn.
- Hệ thống Multiplayer sử dụng Photon Fusion.
- Hệ thống Lobby và quản lý phòng chơi.
- Hệ thống Level Editor.
- Hệ thống lưu và tải màn chơi bằng Prefab.

a. Kết quả Level Design Tool



Hình 3.1: Hình ảnh của Scene Level Design

b. Kết quả lưu Level



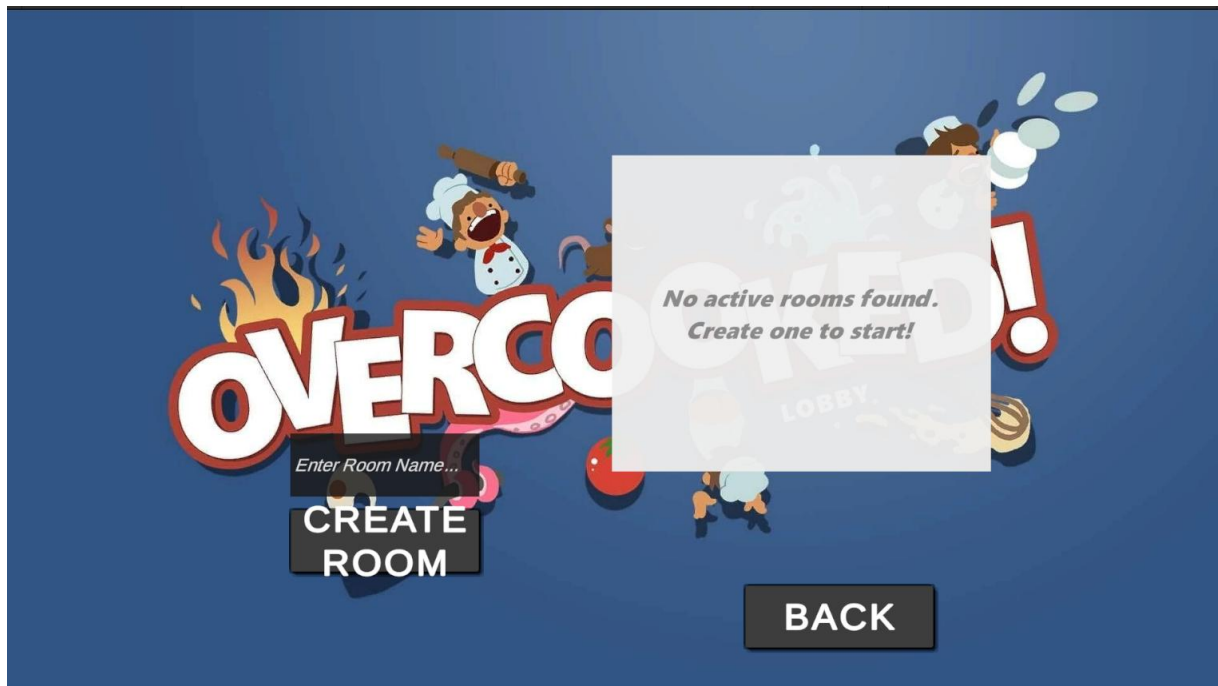
Hình 3.2: Hình ảnh lưu level vào Prefab

c. Hình ảnh Main Menu



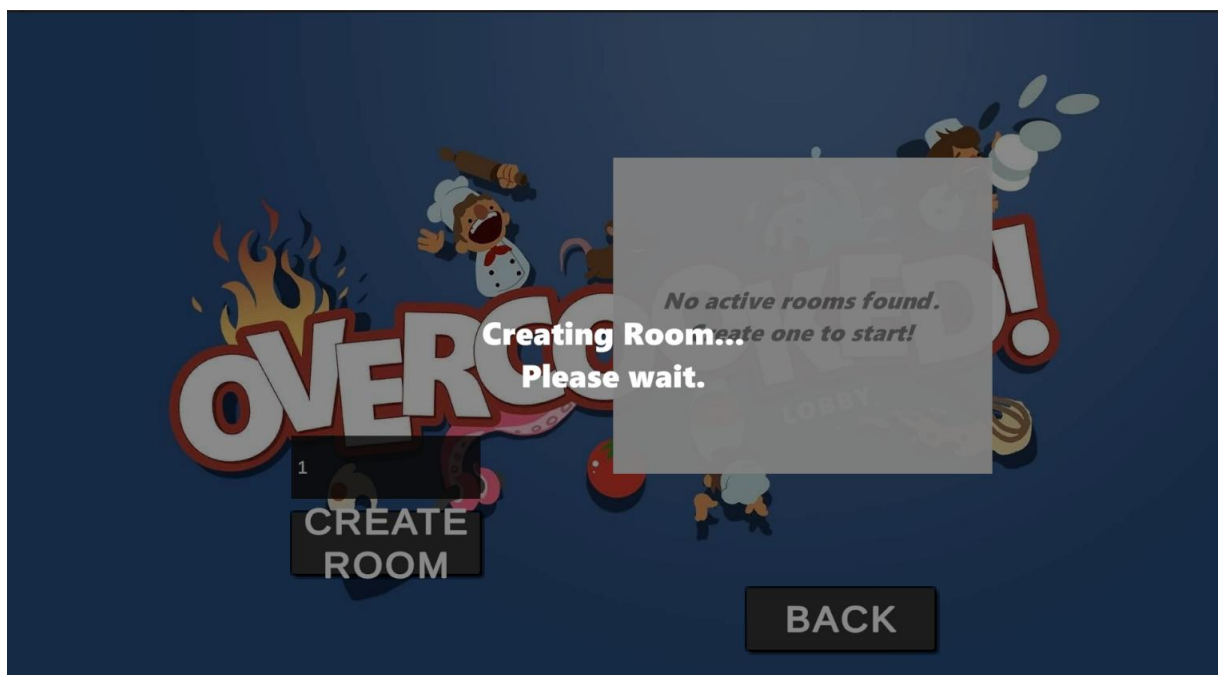
Hình 3.3: Hình ảnh của Screen MainMenu

d. Hình ảnh tạo phòng của chế độ Multiplayer

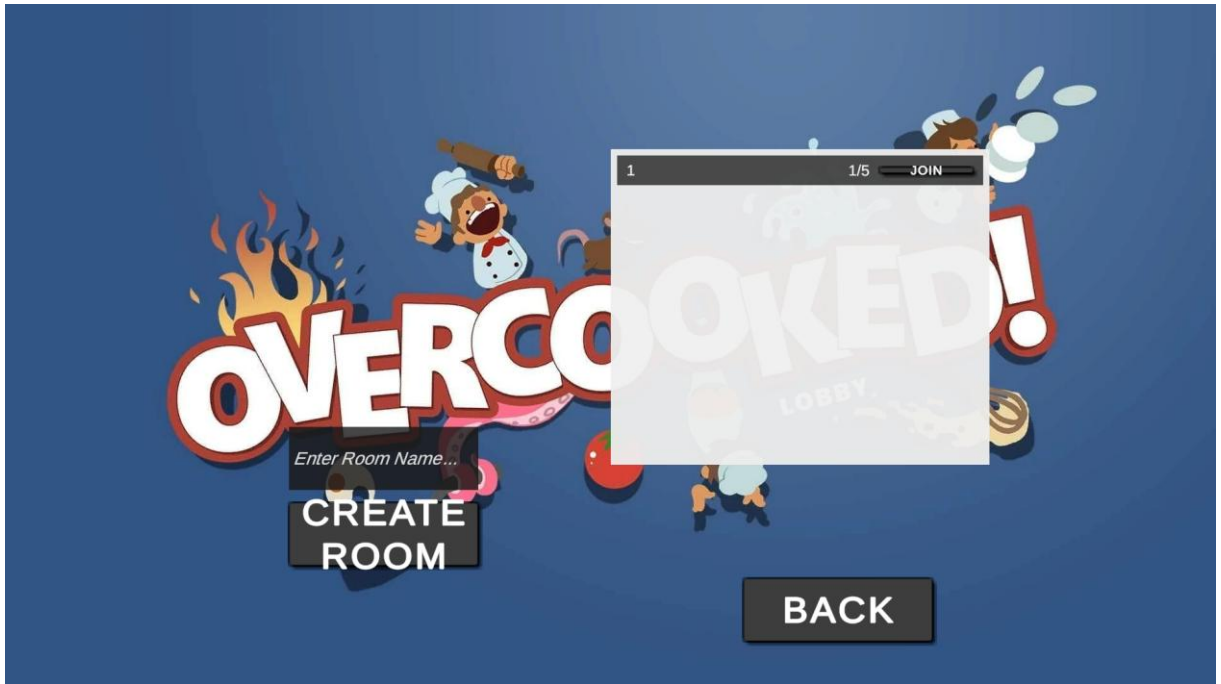


Hình 3.4: Hình ảnh của Screen khi chọn Multiplayer

e. Kết quả chế độ tìm phòng

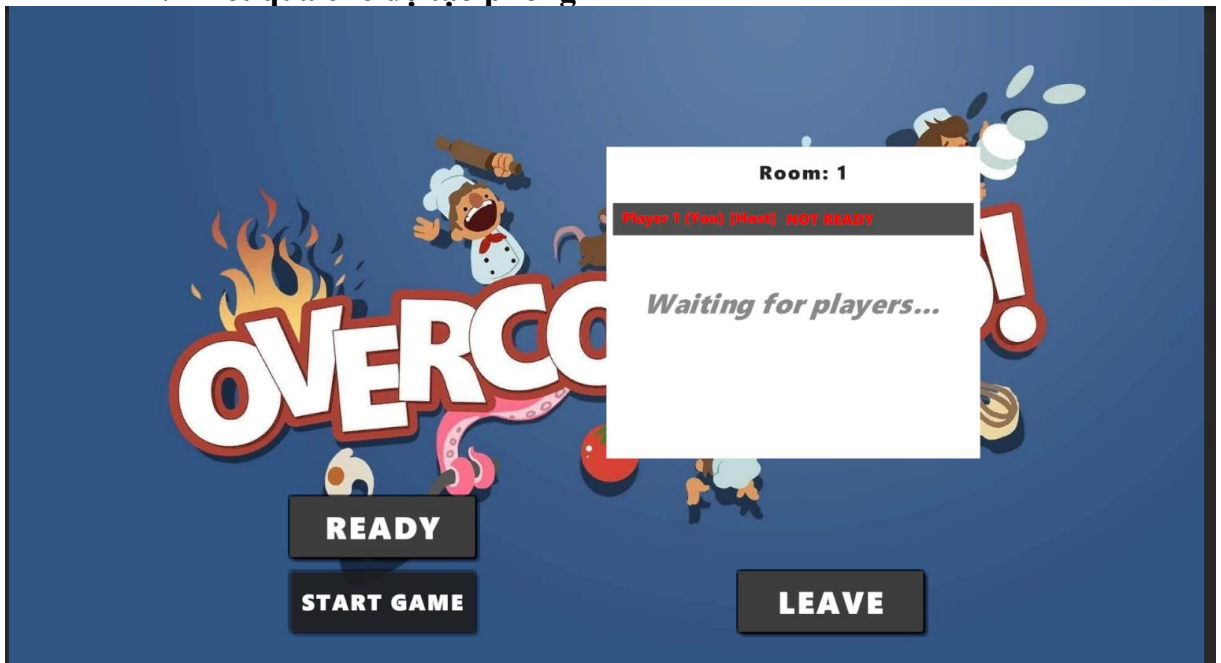


Hình 3.5: Hình ảnh của Screenk khi tạo phòng bằng Photon Fushion

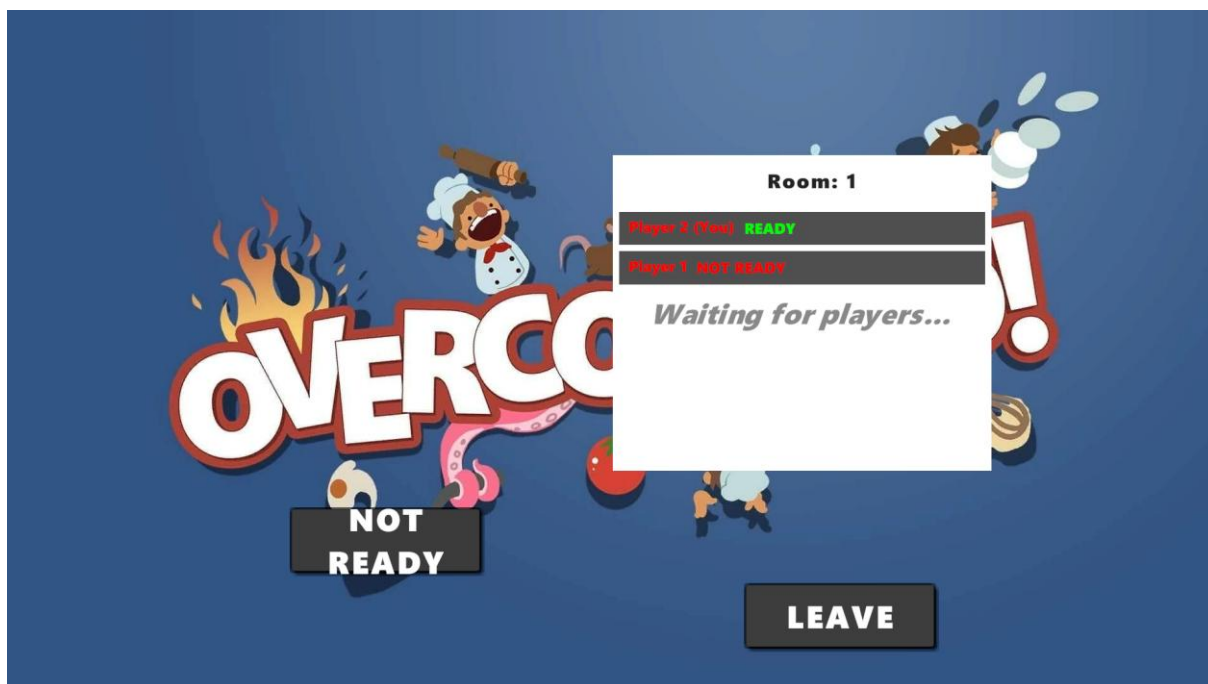


Hình 3.6: Hình ảnh của sảnh chờ vào phòng

f. Kết quả chế độ tạo phòng



Hình 3.7: Hình ảnh khi vào phòng



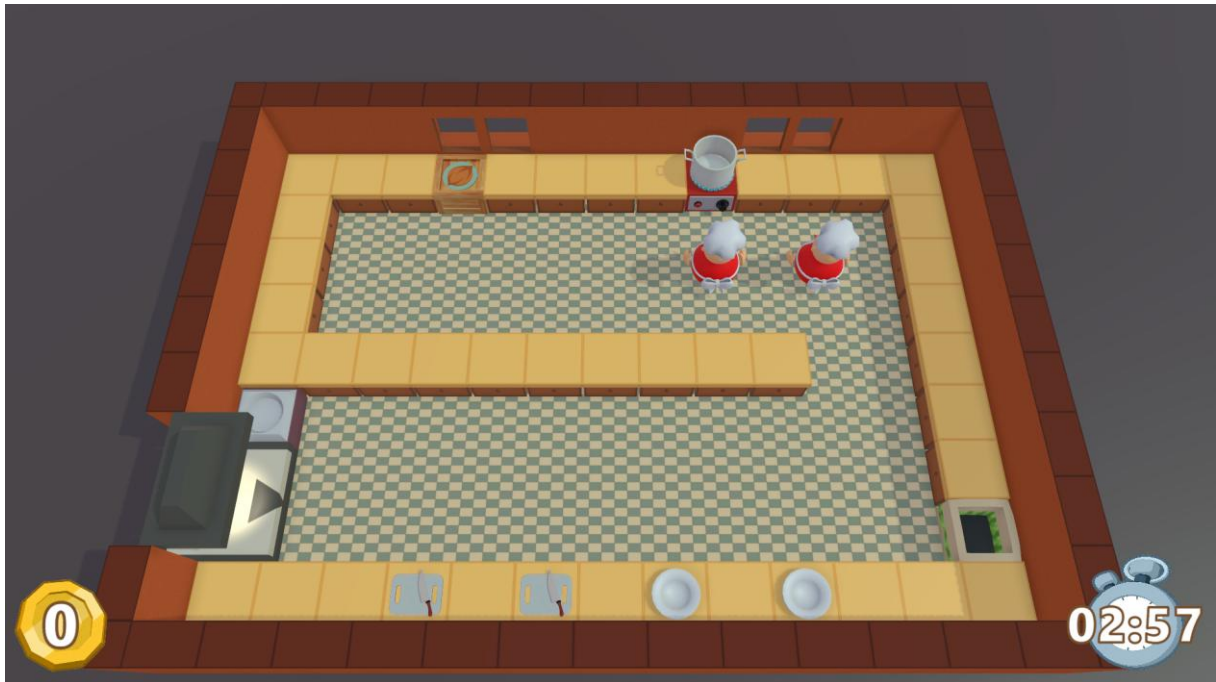
Hình 3.8: Hình ảnh khi vào phòng

g. Kết quả chế độ chơi 1 người



Hình 3.9: Hình ảnh khi vào trò chơi

h. Kết quả chế độ chơi nhiều người



Hình 3.10: Hình ảnh khi vào trò chơi multiplayer

3.10. Kiểm thử chức năng

Bảng kiểm thử chức năng chính của hệ thống:

Chức năng	Kết quả
Tạo phòng chơi	Đạt
Tham gia phòng	Đạt
Đồng bộ người chơi	Đạt
Chế biến món ăn	Đạt
Giao món ăn	Đạt
Lưu màn chơi	Đạt
Tải màn chơi	Đạt

3.11. Đánh giá hệ thống

Sau quá trình triển khai và kiểm thử, hệ thống đã đáp ứng đầy đủ các yêu cầu chức năng đã đề ra trong giai đoạn phân tích.

Về mặt gameplay, người chơi có thể thực hiện đầy đủ các thao tác như di chuyển, thu thập nguyên liệu, sơ chế thực phẩm, nấu ăn và giao món ăn. Các thao tác tương tác được xử lý ổn định và phản hồi nhanh chóng.

Về mặt multiplayer, hệ thống Photon Fusion cho phép nhiều người chơi tham gia cùng một phòng và tương tác trong thời gian thực. Các trạng thái quan trọng như vị trí nhân vật, trạng thái chế biến món ăn và tiến trình màn chơi được đồng bộ chính xác giữa

các máy khách. Việc áp dụng cơ chế State Authority giúp hạn chế xung đột dữ liệu và đảm bảo tính nhất quán của trò chơi.

Về mặt kiến trúc phần mềm, hệ thống được xây dựng dựa trên các nguyên tắc lập trình hướng đối tượng kết hợp với các mẫu thiết kế như Singleton, Observer, Object Pooling và Finite State Machine. Điều này giúp mã nguồn được tổ chức rõ ràng, dễ bảo trì và thuận tiện cho việc mở rộng trong tương lai.

Hệ thống Level Editor hoạt động ổn định và cho phép người dùng tạo mới, chỉnh sửa, lưu và tải màn chơi một cách trực quan. Việc sử dụng định dạng Prefab giúp dữ liệu màn chơi có thể dễ dàng chỉnh sửa và tái sử dụng.

Nhìn chung, các mục tiêu kỹ thuật đặt ra ban đầu đều đã được hoàn thành. Hệ thống có khả năng tiếp tục mở rộng để phát triển thành một sản phẩm hoàn chỉnh hơn trong tương lai.

3.12. Hạn chế

Bên cạnh những kết quả đạt được, quá trình thực hiện đề tài còn tồn tại một số hạn chế nhất định xuất phát từ giới hạn về kinh nghiệm và thời gian phát triển.

a. Hạn chế về hệ thống mạng và chuyển động nhân vật.

Do đây là lần đầu tiên tiếp cận lập trình game online và làm việc với Photon Fusion, nhóm còn thiếu kinh nghiệm trong việc cấu hình và tinh chỉnh các thông số đồng bộ mạng. Cụ thể, chuyển động của nhân vật trong chế độ multiplayer đôi khi chưa thực sự mượt mà, đặc biệt trong điều kiện mạng có độ trễ cao. Mặc dù Photon Fusion đã tích hợp sẵn cơ chế Client-Side Prediction và Reconciliation, việc khai thác tối đa các cơ chế này đòi hỏi kinh nghiệm thực tế nhiều hơn so với những gì nhóm tích lũy được trong thời gian thực hiện đồ án. Ngoài ra, một số tình huống nâng cao như Host Migration khi người tạo phòng mất kết nối đột ngột, hay cơ chế tự động kết nối lại khi Client bị ngắt kết nối giữa chừng, vẫn chưa được xử lý đầy đủ trong phiên bản hiện tại.

b. Hạn chế về chất lượng đồ họa.

Do hạn chế về kỹ năng thiết kế đồ họa 3D và nguồn tài nguyên có sẵn, các mô hình nhân vật, nguyên liệu và môi trường nhà bếp trong trò chơi còn ở mức đơn giản, chưa đạt được tính thẩm mỹ cao. Các hiệu ứng hình ảnh như ánh sáng động, bóng đổ chi tiết hay particle effect trong quá trình nấu ăn chưa được đầu tư kỹ lưỡng, khiến trải nghiệm hình ảnh tổng thể chưa bắt mắt so với các sản phẩm thương mại cùng thể loại. Đây là điểm hạn chế rõ ràng nhất khi so sánh trực quan với Overcooked 2 hay PlateUp!, và là mảng cần được cải thiện đáng kể nếu dự án tiếp tục phát triển.

c. Giới hạn số lượng người chơi.

Hệ thống hiện tại chỉ hỗ trợ tối đa 2 người chơi đồng thời do giới hạn trong quá trình kiểm thử và tối ưu băng thông mạng. Đây là điểm còn khoảng cách so với game gốc Overcooked vốn hỗ trợ đến 4 người chơi, ảnh hưởng đến tính đa dạng và chiều sâu của trải nghiệm hợp tác.

d. Dữ liệu cấu hình chưa tách biệt hoàn toàn khỏi code.

Một số thông số như thời gian nấu, điểm số cộng thêm hay số lượng đơn hàng tối đa vẫn còn hardcoded trực tiếp trong code thay vì được quản lý hoàn toàn qua ScriptableObject. Điều này gây bất tiện khi muốn điều chỉnh cân bằng gameplay mà không cần build lại toàn bộ project.

3.13. Kết luận

Đề tài đã nghiên cứu và xây dựng thành công trò chơi nấu ăn 3D theo phong cách Overcooked trên nền tảng Unity. Hệ thống hỗ trợ chế độ chơi đơn và nhiều người chơi thông qua Photon Fusion, đồng thời áp dụng các kỹ thuật lập trình hướng đối tượng, Observer Pattern, Singleton Pattern, Object Pooling và Finite State Machine nhằm nâng cao khả năng mở rộng và bảo trì hệ thống.

Ngoài ra, đề tài còn phát triển hệ thống Level Editor cho phép thiết kế và lưu trữ màn chơi dưới dạng dữ liệu Prefab, góp phần tăng tính linh hoạt và khả năng mở rộng nội dung trò chơi.

TÀI LIỆU THAM KHẢO

- [1] Unity Technologies, *Unity User Manual*, Unity Technologies, San Francisco, CA, USA, 2026. [Online]. Available: <https://docs.unity3d.com/Manual/index.html>. [Accessed: Jun. 1, 2026].
- [2] Unity Technologies, *Unity Scripting API Documentation*, Unity Technologies, 2026. [Online]. Available: <https://docs.unity3d.com/ScriptReference/>. [Accessed: Jun. 1, 2026].
- [3] Exit Games, *Photon Fusion Documentation*, Exit Games GmbH, Hamburg, Germany, 2026. [Online]. Available: <https://doc.photonengine.com/fusion/current/getting-started/overview>. [Accessed: Jun. 1, 2026].
- [4] Microsoft, *C# Documentation*, Microsoft Corporation, Redmond, WA, USA, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/csharp/>. [Accessed: Jun. 1, 2026].
- [5] R. Nystrom, *Game Programming Patterns*. Scotts Valley, CA, USA: Genever Benning, 2014.
- [6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [7] M. Gregory, *Game Engine Architecture*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2018.
- [8] J. Schell, *The Art of Game Design: A Book of Lenses*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2019.
- [9] Ghost Town Games, *Overcooked! 2*, Team17 Digital Ltd., 2018. [Online]. Available: <https://www.team17.com/games/overcooked-2/>. [Accessed: Jun. 1, 2026].
- [10] Yogscast Games, *PlateUp!*, Yogscast Games, 2022. [Online]. Available: <https://www.plateupgame.com/>. [Accessed: Jun. 1, 2026].
- [11] JetBrains, *JetBrains Rider Documentation*, JetBrains s.r.o., 2026. [Online]. Available: <https://www.jetbrains.com/rider/documentation/>. [Accessed: Jun. 1, 2026].
- [12] Microsoft, *Visual Studio Code Documentation*, Microsoft Corporation, 2026. [Online]. Available: <https://code.visualstudio.com/docs>. [Accessed: Jun. 1, 2026].
- [13] R. Keith, *Multiplayer Game Programming: Architecting Networked Games*. Indianapolis, IN, USA: Wiley Publishing, 2018.
- [14] Unity Technologies, *ScriptableObject Manual*, Unity Technologies, 2026. [Online]. Available: <https://docs.unity3d.com/Manual/class-ScriptableObject.html>. [Accessed: Jun. 1, 2026].
- [15] Unity Technologies, *Prefab Workflow Documentation*, Unity Technologies, 2026. [Online]. Available: <https://docs.unity3d.com/Manual/Prefabs.html>. [Accessed: Jun. 1, 2026].